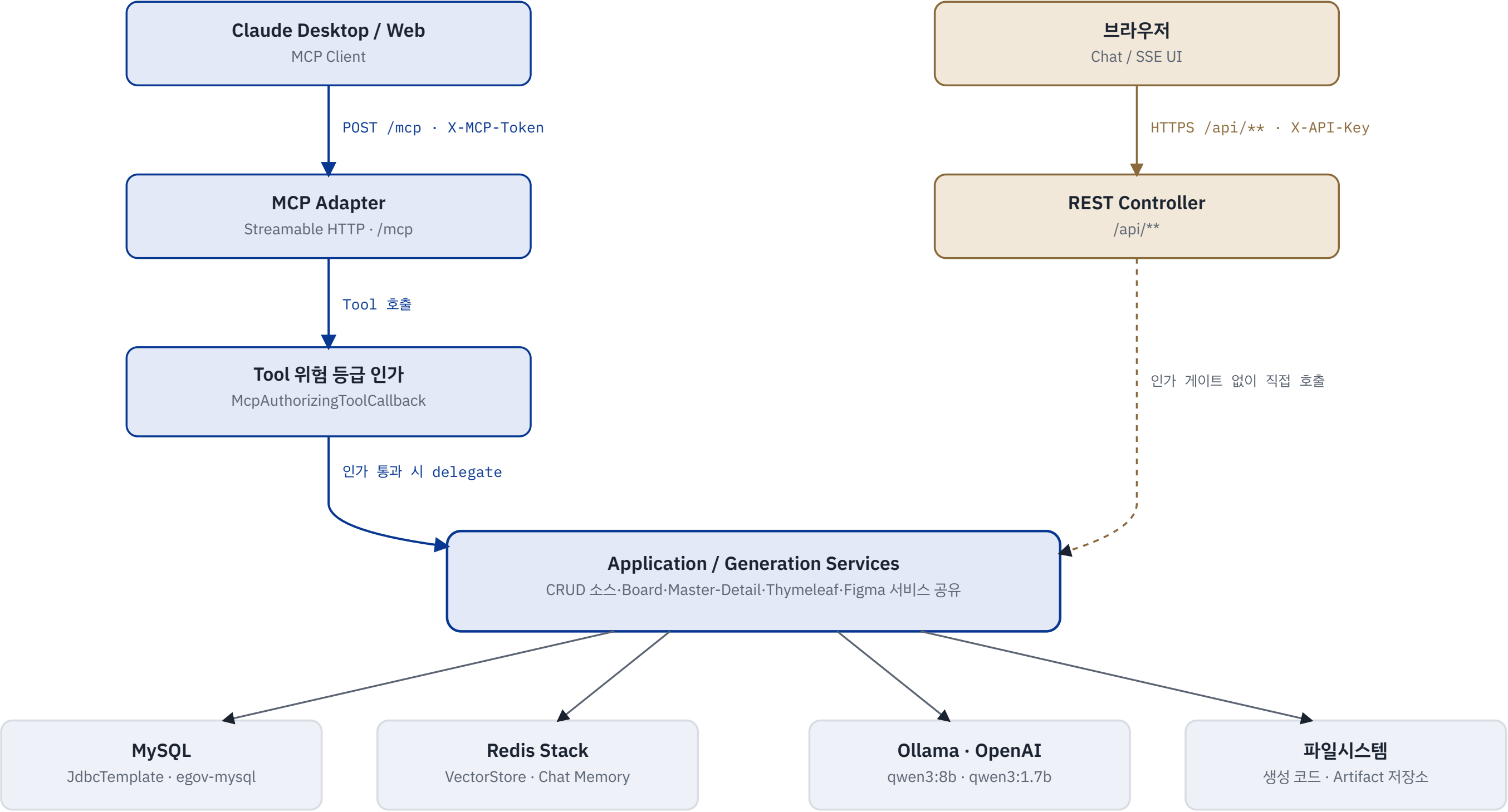


프로젝트 전체 아키텍처

Spring Boot 단일 서버가 `/mcp` (Streamable HTTP)와 `/api/**` (REST) 두 진입점을 같은 서비스 계층으로 연결하고, Figma 디자인 시스템과 로컬 Thymeleaf/JSP 화면을 별도 Node.js 서브 프로젝트 6개가 JSON 계약으로 오가며 동기화합니다.

1. 런타임 요청 흐름

MCP Tool 호출과 REST 호출이 같은 Application/Generation 서비스로 합류하되, 인가 경로는 다릅니다.



— MCP 경로 (Tool 인가 필수) — REST 경로 (API Key만)

MCP와 REST 모두 결국 같은 **Application / Generation Services**로 합류하지만, 인가 경로가 다릅니다. MCP Tool 호출은 McpAuthorizingToolCallback이 매 호출마다 위험 등급을 확인한 뒤에만 서비스로 위임되는 반면, REST Controller는 X-API-Key 인증만 통과하면 이 게이트를 거치지 않고 서비스를 직접 호출합니다. 두 진입점의 신뢰 경계 차이를 보여주는 것이 이 다이어그램의 핵심입니다.

2. 생성 파이프라인과 지원 자산

화면 하나를 만드는 데 관여하는 Figma 파이프라인과 코드 생성 Tool 체인, 그리고 이것들과는 완전히 독립된 채팅(RAG) 기능을 한 섹션에 모았습니다. 세 개의 하위 다이어그램은 서로 다른 관심사를 다루므로 개별적으로 읽으십시오.

2.1 Figma ↔ Thymeleaf 디자인 파이프라인

Figma 파일을 허브로, 읽기(디자인→코드)·캡처(코드→디자인 재현)·내보내기·토큰 동기화 네 갈래가 별도 Node.js 프로젝트를 오갑니다.

목표 화면 생성 파이프라인

Figma의 시각 의미를 내부 계약으로 정규화한 뒤 업무 계약과 결합하고, 승인된 명세만 코드 생성과 검증 단계로 보냅니다. 이 흐름은 Figma 화면을 HTML로 그대로 복제하는 변환기가 아닙니다.



청색 — Figma에서 유래한 시각 계약 갈색 — Controller·VO·DB·권한 업무 Binding 녹색 — 승인 및 검증을 통과해야 하는 Gate

Figma가 제어하는 것

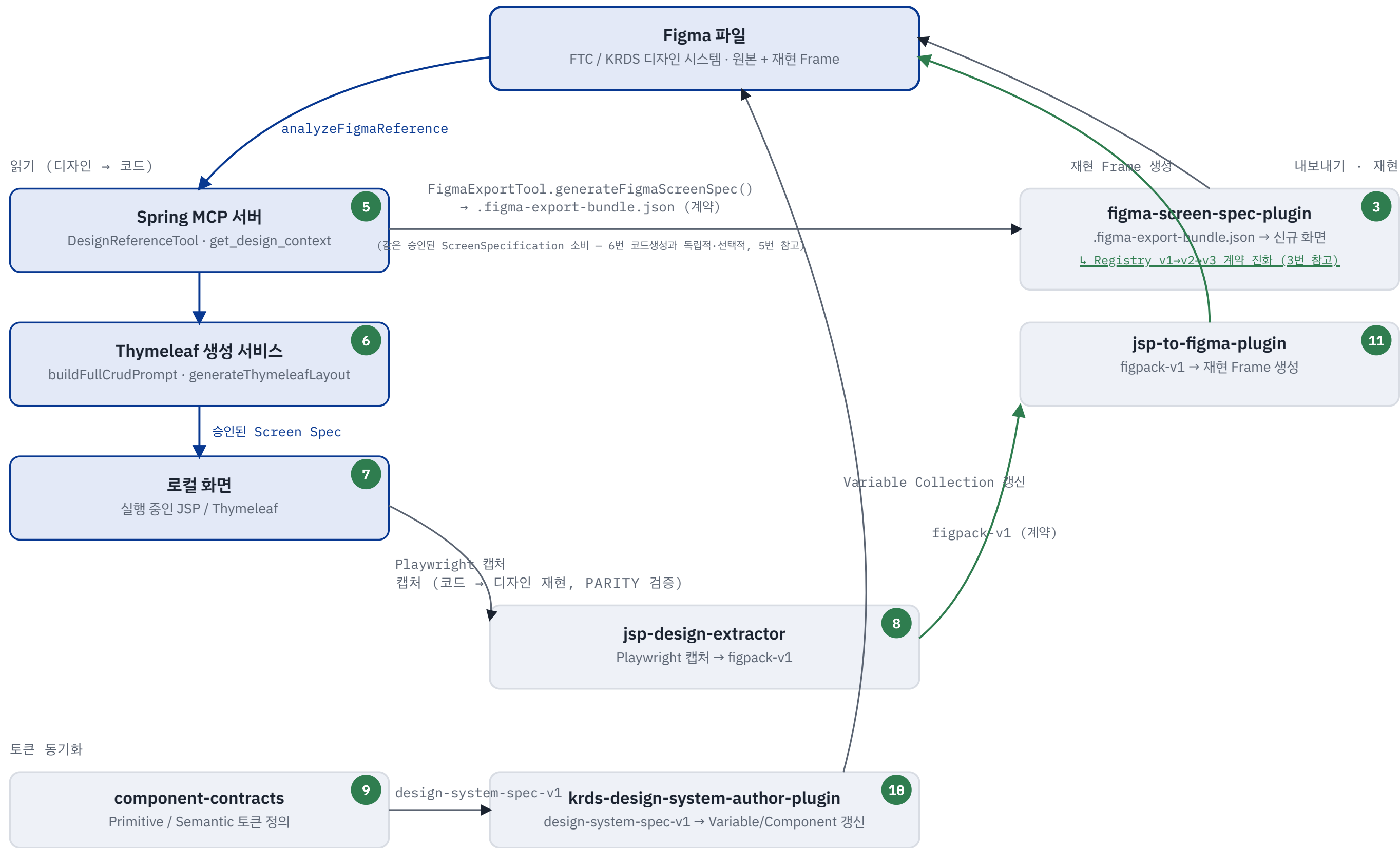
시각 구조, 레이아웃, 디자인 토큰, 승인된 컴포넌트 선택

업무 Binding의 기준

Controller·VO·DB Schema·권한과 APPROVED ScreenSpecification

승인·검증 Gate의 기준

APPROVED 상태와 Binding Audit·Build·Desktop/Tablet/Mobile Render 통과



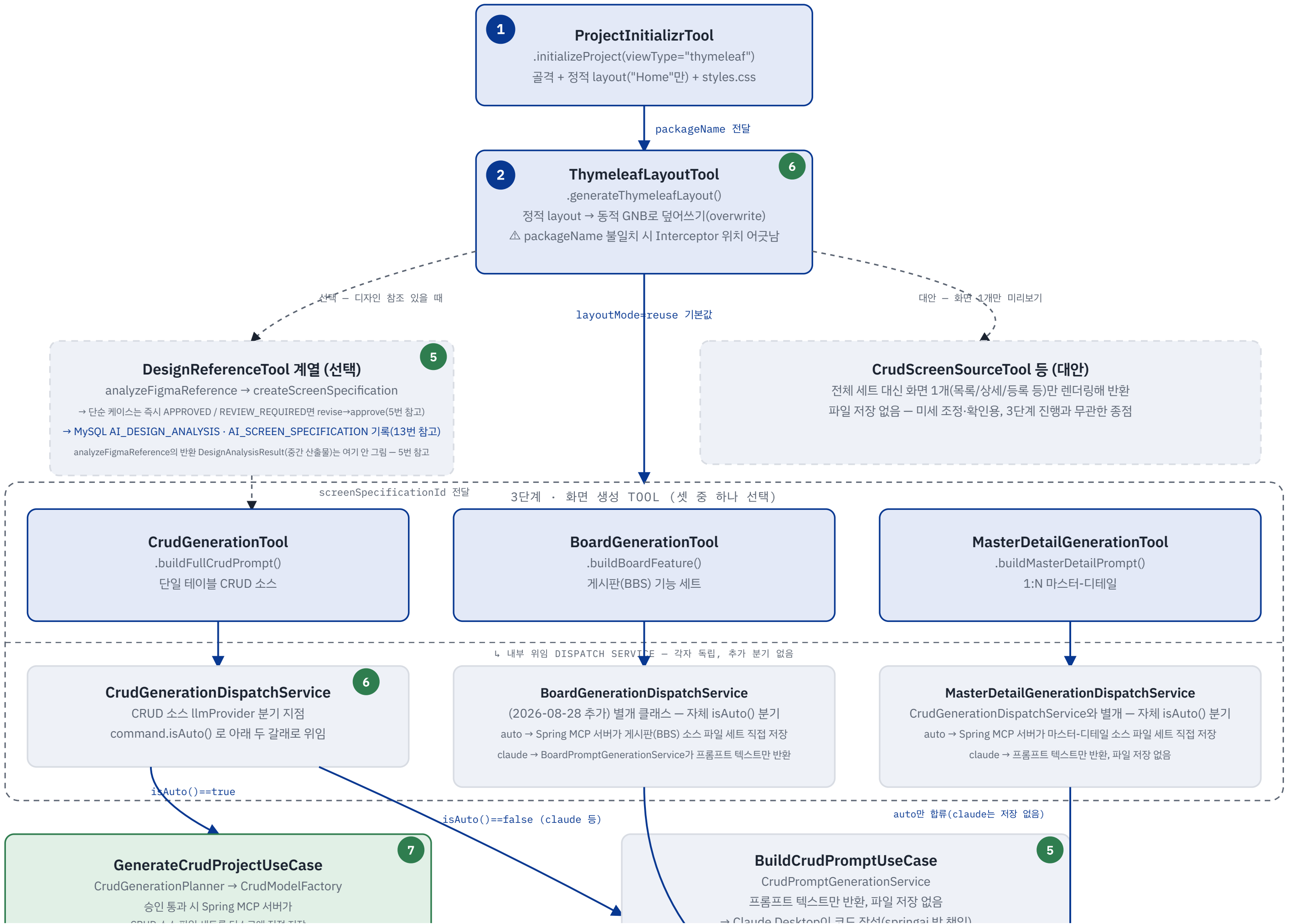
— 읽기: 디자인 → 코드 생성 — 캡처: 코드 → 재현 (parity 검증용) — 내보내기 · 토큰 동기화

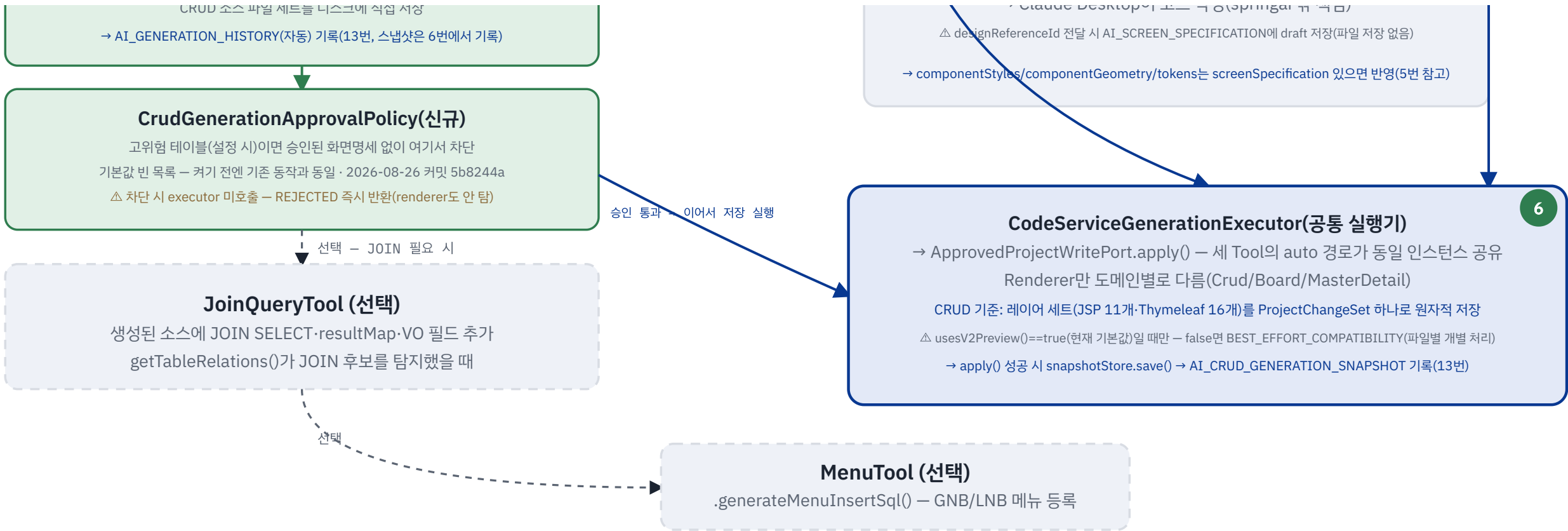
모든 화살표가 website-figma-contract의 JSON Schema(figpack-v1, .figma-export-bundle.json, design-system-spec-v1)를 계약으로 삼아 서로 다른 Node.js 프로젝트를 오갑니다. **읽기** 경로는 Figma 원본을 읽어 Thymeleaf 화면을 생성하고, **캡처** 경로는 그 결과 화면을 Playwright로 다시 찍어 Figma에 재현 Frame으로 되돌려 원본과 나란히 시각 비교(parity 검증)할 수 있게 합니다. **내보내기**와 **토큰 동기화** 경로는 각각 신규 화면 생성과 디자인 토큰 갱신을 담당하는 별도 흐름입니다. 내보내기 화살표는 정확히는 5번에서 승인된 ScreenSpecification을 받은

`FigmaExportTool.generateFigmaScreenSpec()`에서 시작합니다 — `FigmaExportBundleAssembler`가 6개 조각 (`figmaScreenSpec·designSystemProfile·componentRegistry·resolvedComponentRegistry·screenPattern·variantRuleSet`)을 조립하고, `FigmaExportController`의 `GET /screens/{screenId}/download` 엔드포인트가 파일명 `{screenId}.figma-export-bundle.json`으로 내려 줍니다. 이때 `generateFigmaScreenSpec()`이 먼저 MySQL `AI_FIGMA_SCREEN_SPEC`(PK: `SCREEN_ID`, `SCREEN_VERSION`, `revision-append`)에 기록 한 뒤 조립을 시작합니다 — 13번 다이어그램 참고. **Thymeleaf 코드 생성(6번 경로)과는 완전히 독립된 별도 소비 경로**이며, `figma-screen-spec-plugin`이 이를 검증한 뒤 실제 Component Instance로 화면을 생성합니다. `figma-screen-spec-plugin` 박스의 **3번 배지**를 클릭하면 이 `.figma-export-bundle.json`이 어떤 계약 버전(`Catalog/Registry v1→v2→v3`)을 거쳐 만들어지는지 보여주는 3번 다이어그램으로 바로 이동합니다. 나머지 7개 박스에 붙은 **5~11번 배지**는 각 구성요소 내부가 실제로 어떤 단계를 거치는지 보여주는 전용 다이어그램으로 이동합니다.

2.2 코드 생성 Tool 오케스트레이션

화면 하나를 만들기까지 사람이 순서대로 호출해야 하는 MCP Tool 체인입니다. 아래는 파일 위치가 아니라 **Tool 실행 순서와 서로의 전제 조건**을 보여줍니다 — 특히 1·2단계는 같은 layout 파일 경로를 두고 "정적 골격 → 동적 업그레이드" 관계로 순서가 강제됩니다.





Legend: — Tool·서비스·선택 단계 (dashed line) — Crud auto 경로 - 실제 파일 저장 + AI_GENERATION_HISTORY·AI_CRUD_GENERATION_SNAPSHOT DB 기록이 일어나는 지점 (solid line)

1·2단계는 같은 **layout 파일 경로**(`templates/layout/{default,gnb,lnb,breadcrumb,footer}.html`)를 두고 순서가 강제됩니다 — 1단계가 만드는 `gnb.html`은 메뉴가 "Home" 하나뿐인 정적 자리채움이고, 2단계가 `overwriteLayout=true`(기본값)로 이를 `LETTNMENUINFO` 기반 동적 GNB로 완전히 덮어 씹니다. `generateThymeleafLayout()`의 Tool 설명 자체가 "packageName은 initializeProject()에 전달했던 것과 반드시 동일해야 한다"고 명시합니다 — 다르면 `EgovGnbMenuInterceptor`가 실제 CRUD 소스 패키지와 어긋난 위치에 생성되어 동작하지 않습니다. 3단계 이후가 이번에 새로 밝혀진 진짜 분기 지점이며, **세 Tool 모두 각자의 분기점을 갖되 서로 공유하지 않습니다** — `CrudGenerationTool`은 `CrudGenerationDispatchService`로, `MasterDetailGenerationTool`은 이름은 비슷하지만 **완전히 별개인** `MasterDetailGenerationDispatchService`로, `BoardGenerationTool`도 **(2026-08-28 추가)** 마찬가지로 완전히 별개인 `BoardGenerationDispatchService`로 갑니다(세 클래스의 JavaDoc이 각자 "IImProvider(auto vs claude/ 기타) 분기가 존재하는 유일한 지점"이라는 같은 문구를 씹니다 — "유일한"은 각 클래스 내부 기준이지, 세 Tool이 같은 분기점을 쓴다는 뜻이 아닙니다). 세 경로 모두 auto면 **Spring MCP 서버**가 각자의 소스 파일 세트(CRUD는 VO/Controller/Service/Mapper/View, MasterDetail은 1:N 마스터-디테일 레이어, Board는 게시판(BBS) 기능 세트)를 직접 저장해 7번 다이어그램의 로컬 화면이 되고, claude 등이면 서버는 프롬프트 텍스트만 돌려주고 **코드를 실제로 쓰는 주체가 springai에서 Claude Desktop으로 바뀝니다**(Board의 claude 경로는 `BoardPromptGenerationService`가 테이블 구성·스키마·업무규칙·프로그램 메타데이터를 텍스트로 조립합니다). MasterDetail·Board의 UseCase들은 전용 다이어그램이 없어 이 박스 안에 요약만 표시했습니다. `ScreenSourceTool` 계열(3단계 위 대안)과 `JoinQueryTool`(CRUD auto 경로 이후 선택)은 메인 흐름을 보조하는 별도 진입점입니다.

(2026-08-26 갱신) 위 다이어그램에 `CrudGenerationApprovalPolicy`(신규) 박스가 추가된 대로, `CrudGenerationPlanner`(7번 `GenerateCrudProjectUseCase` 내부) 안에 조건부 승인 게이트가 추가됐습니다(커밋 5b8244a) — `CrudGenerationApprovalPolicy`가 `app.crud-generation.approval-required-tables`(테이블 목록) 또는 `approval-required-for-all`(전체 강제) 설정을 확인해, 지정된 테이블은 `viewType`과 무관하게 승인된 화면명세(`designReferenceId/screenSpecificationId`) 없이는 `crudSchemaQueryService.fetchColumns()` 호출 전에 즉시 차단합니다. 기존 `V2_APPLY+Thymeleaf` 전용 화면명세 필수화 가드와 별개이며, 이 가드는 JSP 뷰에도 적용될 수 있다는 점이 다릅니다. 기본값은 빈 목록·`false`라 켜기 전까지는 기존 동작과 동일합니다. 차단 시 `AI_GENERATION_OPERATION_AUDIT`에 `REJECTED/ failureStage=approval-policy`로 기록되어 14.1절 API로 조회됩니다. 배경은 docs/architecture/CRUD_명시적_승인_단계_도입_타당성_검토.md 참고 — 검토 당시 옵션 A(Thymeleaf식 3단계 `preview/approve/apply`)도 함께 검토했으나, 기존 Tool 계약을 깨는 변경이라 채택하지 않았습니다.

(2026-08-26 갱신, 2) `GenerateCrudProjectUseCase` 박스 아래로 내려가는 "내부 호출" 화살표는 `CrudGenerationApprovalPolicy`와 `CodeServiceGenerationExecutor` (공동 실행기) 두 곳 다 **같은 의미**입니다 — 순서대로 실행되는 별도 단계가 아니라 `CrudGenerationApplicationService.execute()`(실제 `GenerateCrudProjectUseCase` 구현체) 메서드 하나가 내부에서 `planner.plan()`

(ApprovalPolicy 포함) → `renderer.render()` → `executor.execute()`(CodeServiceGenerationExecutor) → `historyRecorder.record()`
(AI_GENERATION_HISTORY) 순서로 호출하는 것을 확대해서 보여주는 것입니다. 즉 GenerateCrudProjectUseCase 박스의 "Spring MCP 서버가 CRUD 소스 파일 세트를 디스크에 직접 저장 → AI_GENERATION_HISTORY 기록"과 아래 CodeServiceGenerationExecutor 박스의 "→ ApprovedProjectWritePort.apply()"는 **같은 한 번의 저장 동작**을 서로 다른 확대 수준에서 설명한 것이지, 두 번 저장한다는 뜻이 아닙니다.

(2026-08-26 갱신, 3) CrudGenerationApprovalPolicy → CodeServiceGenerationExecutor 관계는 두 갈래로 갈라지는 분기가 아니라 **단일 순차 게이트**입니다 — CrudGenerationPlanner .plan()의 실제 조건은 approvalPolicy.requiresApproval(tableName) && isBlank(designReferenceId) && isBlank(screenSpecificationId)이며, 이 조건이 **참이면 그 자리에서 CrudGenerationPlan.rejected()**를 반환하고 끝납니다 — crudSchemaQueryService.fetchColumns()조차 호출되지 않고, `renderer.render().executor.execute()`(CodeServiceGenerationExecutor)는 당연히 실행되지 않습니다. 조건이 거짓이면(고위험 테이블이 아니거나, designReferenceId·screenSpecificationId 중 하나라도 값이 있으면) 이 게이트는 그냥 통과하고 그대로 직진해 결국 CodeServiceGenerationExecutor에 도달합니다. 단, 이 게이트는 ID가 **비어있는지만** 보며, 그 ID가 실제로 APPROVED 상태인지는 검사하지 않습니다 — 실제 승인 상태 검증은 몇 줄 뒤 generationDesignContextService.resolve(...) 호출에서 별도로 이루어지는, 이 게이트와는 다른 실패 지점입니다.

(2026-08-26 갱신, 4) CodeServiceGenerationExecutor가 실제로 저장하는 내용을 CrudLayerDefinition 소스로 확인했습니다 — CrudGenerationRenderer가 렌더링에 성공한 레이어들을 **파일 하나씩이 아니라 ProjectChangeSet 하나로 묶어 한 번에** ApprovedProjectWritePort.apply()에 넘깁니다(원자적 저장). CRUD 기준 레이어 구성은 다음과 같습니다.

- 공통 7개(JSP·Thymeleaf 동일): {Domain}VO.java·{Domain}Mapper.java·{Domain}Mapper.xml·{Domain}Service.java·Egov{Domain}ServiceImpl.java·Egov{Domain}Controller.java·Egov{Domain}ValidationHandler.java
- + JSP 전용 4개 = **총 11개**: List.jsp·Detail.jsp·Regist.jsp·Updt.jsp
- + Thymeleaf 전용 9개 = **총 16개**: layout 5종(default·gnb·lnb·breadcrumb·footer.html) + List.html·Detail.html·Regist.html·Updt.html

"JSP 11개 / Thymeleaf 16개"는 CLAUDE.md에 이미 명시된 수치와 정확히 일치합니다. Board·MasterDetail은 각자의 BoardLayerDefinition·MasterDetailLayerDefinition이 정의하는 별도 레이어 세트를 같은 방식(원자적 ProjectChangeSet)으로 저장합니다.

항목	CRUDGENERATIONTOOL	BOARDGENERATIONTOOL	MASTERDETAILGENERATIONTOOL
분기(Dispatch)	<code>CrudGenerationDispatchService</code> — <code>isAuto()</code> 분기 있음	<code>BoardGenerationDispatchService</code> (2026-08-28 추가) — CRUD와 별개 클래스지만 동일한 패턴의 <code>isAuto()</code> 분기	<code>MasterDetailGenerationDispatchService</code> — CRUD와 별개 클래스지만 동일한 패턴의 <code>isAuto()</code> 분기
auto 저장 주체	<code>CodeServiceGenerationExecutor</code> — <code>GenerationExecutor</code> 구현체가 코드베이스에 이것 하나뿐이라 세 Tool이 같은 Spring 싱글톤을 공유		
claude 경로	<code>BuildCrudPromptUseCase</code> — 프롬프트만 반환, 파일 저장 없음	<code>BuildBoardPromptUseCase</code> (<code>BoardPromptGenerationService</code> , 2026-08-28 추가) — 테이블 구성·스키마·업무규칙 프롬프트만 반환	<code>BuildMasterDetailPromptUseCase</code> — 프롬프트만 반환
렌더러(저장 내용 결정)	<code>CrudGenerationRenderer</code> → <code>CrudTemplateRenderer</code>	<code>BoardGenerationRenderer</code> → <code>BoardTemplateRenderer</code>	<code>MasterDetailGenerationRenderer</code> → <code>MasterDetailTemplateRenderer</code>
레이어 정의	<code>CrudLayerDefinition</code> (VO/Controller/Service/Mapper/View)	<code>BoardLayerDefinition</code> (BBS 전용 레이어)	<code>MasterDetailLayerDefinition</code> (1:N 마스터-디테일 전용 레이어)
실제 파일 쓰기	<code>ApprovedProjectWritePort.apply()</code> (공통)		
이력 기록	<code>GenerationHistoryRecorder</code> (공통 클래스)		

핵심 발견: "저장 주체"는 사실상 하나입니다. 분기 로직은 세 Tool이 서로 다르지만(위 표 1행), 실제로 파일을 쓰는 실행기·Write Port는 CodeServiceGenerationExecutor→ ApprovedProjectWritePort 하나로 합류합니다(2·6·7행). 도메인별 차이는 "누가 저장 하느냐"가 아니라 "무엇을 저장하느냐"에 있습니다 — 서로 다른 Renderer가 서로 다른 .ftl 템플릿 세트로 파일 목록만 다르게 만들고, 그 결과물만 같은 실행기에 넘깁니다.

항목	CRUDGENERATIONTOOL	MASTERDETAILGENERATIONTOOL	BOARDGENERATIONTOOL
역할 슬롯 수	1개(<code>tableName</code>)	2개(<code>masterTable</code> , <code>detailTable</code>)	5개(게시글/마스터/사용권한/파일/파일상세)
테이블명 자유도	자유(필수 파라미터)	자유(둘 다 필수 파라미터, 하드코딩 없음)	자유(단, <code>@Nullable</code> 이라 미지정 시 <code>LETTN*</code> 기본값으로 대체)
<code>llmProvider</code> 파라미터	있음	있음	있음(2026-08-28 추가)
auto/claude 분기	<code>CrudGenerationDispatchService</code> 에서 <code>isAuto()</code>	<code>MasterDetailGenerationDispatchService</code> 에서 자체 <code>isAuto()</code> (CRUD와 별개 구현체)	<code>BoardGenerationDispatchService</code> 에서 자체 <code>isAuto()</code> (CRUD와 별개 구현체, 2026-08-28 추가)
반환값 구조	<code>Orchestrated</code> (저장) / <code>Prompted</code> (텍스트만)	동일하게 <code>Orchestrated</code> / <code>Prompted</code> 2종	동일하게 <code>Orchestrated</code> / <code>Prompted</code> 2종(<code>BoardToolResult</code> , 2026-08-28 추가)

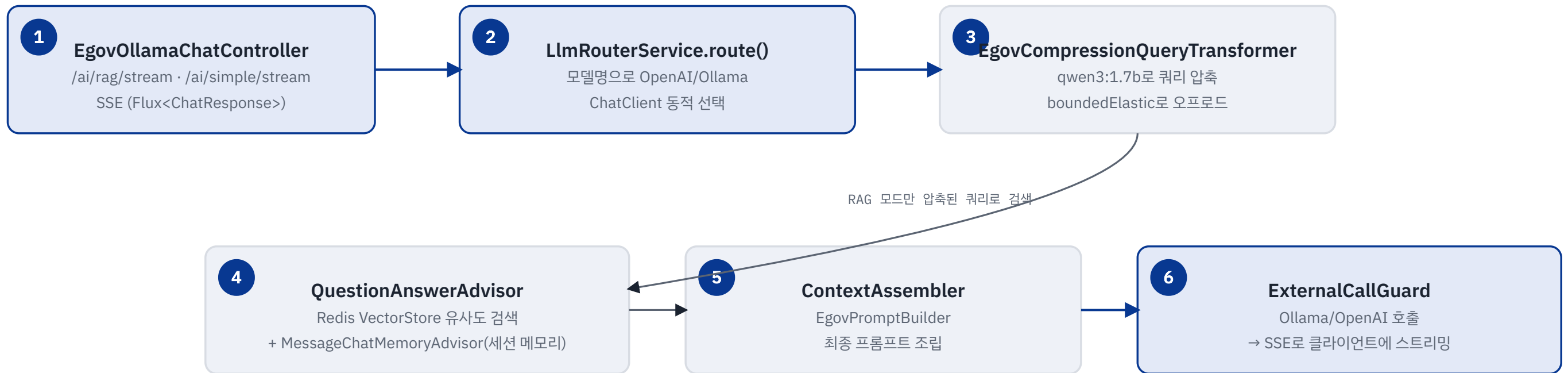
세 Tool 최종 비교. 위 표(분기·렌더러·저장 주체 비교)가 "누가/무엇을 저장하는가"에 초점을 맞췄다면, 이 표는 **Tool 호출 계약(파라미터·분기 유무·반환 타입)** 관점의 비교다. `MasterDetailGenerationTool`은 역할 슬롯이 1개→2개로 늘어났을 뿐 `llmProvider` 분기와 `Orchestrated/Prompted` 반환 구조를 그대로 계승한 **`CrudGenerationTool`의 확장판**에 가깝다. `BoardGenerationTool`은 원래 처음부터 결정론적 파이프라인만으로 설계돼 이 계열에서 벗어나 있었으나, **(2026-08-28 갱신)** 검토 문서 (`BoardGenerationTool_llmProvider_도입_검토.md`) → 구현계획(`BoardGenerationTool_llmProvider_구현계획.md`)을 거쳐 `llmProvider` 분기가 추가되면서, 이제 셋 다 같은 계열(역할 슬롯 개수만 다르고 분기·반환 구조는 동일한 패턴)에 속한다. "테이블명 자유도" 행의 `LETTN*`은 하드코딩된 고정 테이블이 아니라 `BoardTableSetResolver`가 파라미터 미지정 시에만 적용하는 기본값이며, 사용자가 임의 테이블명을 넘기면 그 이름 그대로 `INFORMATION_SCHEMA.COLUMNS`를 조회해 생성한다 — 고정된 것은 테이블 "이름"이 아니라 "역할 슬롯 개수"라는 구조다.

항목	AUTO	CLAUDE
누가 코드를 만드나	Spring MCP 서버(springai) 자신이 내부적으로 FreeMarker 템플릿을 렌더링	Claude Desktop(호출한 대화 상대)이 직접 코드를 작성
서버가 하는 일	DB 스키마 조회 → 렌더링 → 파일을 디스크에 직접 저장까지 전부 수행	스키마 정보 + 작성 지시가 담긴 프롬프트 텍스트만 반환하고 끝
파일 저장	서버가 즉시 저장(<code>CodeServiceGenerationExecutor</code> → <code>ApprovedProjectWritePort</code>)	저장 없음 — 이후 Claude가 별도 Tool(<code>CodeSaverTool.saveCode()</code> 등)로 저장해야 함
토큰 소비	적음(LLM 개입 없이 결정론적으로 생성)	많음(Claude가 프롬프트를 읽고 코드를 직접 생성)
반환 타입	<code>Orchestrated</code>	<code>Prompted</code>

"auto/claude 분기"의 의미. 화면 생성 Tool을 호출할 때 넘기는 `llmProvider` 파라미터 값에 따라 **누가 실제로 소스 코드를 만들어서 저장하는지**가 완전히 달라지는 갈림길이다. 이 분기가 실제로 일어나는 위치는 Tool마다 다른 서비스다 — `CrudGenerationTool`은 `CrudGenerationDispatchService`가 `command.isAuto()`로 분기하고, `MasterDetailGenerationTool`은 이름은 비슷하지만 **완전히 별개 클래스**인 `MasterDetailGenerationDispatchService`가 자체적으로 같은 패턴의 분기를 수행한다. `BoardGenerationTool`도 **(2026-08-28 추가)** 마찬가지로 완전히 별개 클래스인 `BoardGenerationDispatchService`가 자체적으로 같은 패턴의 분기를 수행한다(도입 전에는 이 분기 자체가 없어 항상 서버가 직접 저장하는 경로로만 동작했다). 즉 이 스위치는 "이 Tool 호출 결과로 서버가 파일을 즉시 써버릴지, 아니면 Claude에게 작성을 넘길지"를 결정하며, 이제 세 Tool 모두 이 스위치를 갖고 있다.

2.3 RAG / Chat 흐름

Figma·Thymeleaf 생성과는 완전히 독립된 별도 기능입니다. Ollama·OpenAI 채팅에 Redis 기반 RAG를 결합합니다.



3단계 쿼리 압축은 `rag.enable-query-compression` 설정이 켜져 있고 /ai/rag/stream 경로일 때만 실행되며, Ollama 동기 호출이라 HTTP 요청 스레드를 막지 않도록 `boundedElastic` 스레드로 오프로드합니다. /ai/simple/stream은 2단계에서 바로 6단계로 건너뛰어 RAG 없이 순수 채팅만 수행합니다. 문서 인제스트(PDF → 임베딩 → Redis 저장)는 RagTool이 별도로 담당하며 이 다이어그램의 범위 밖입니다.

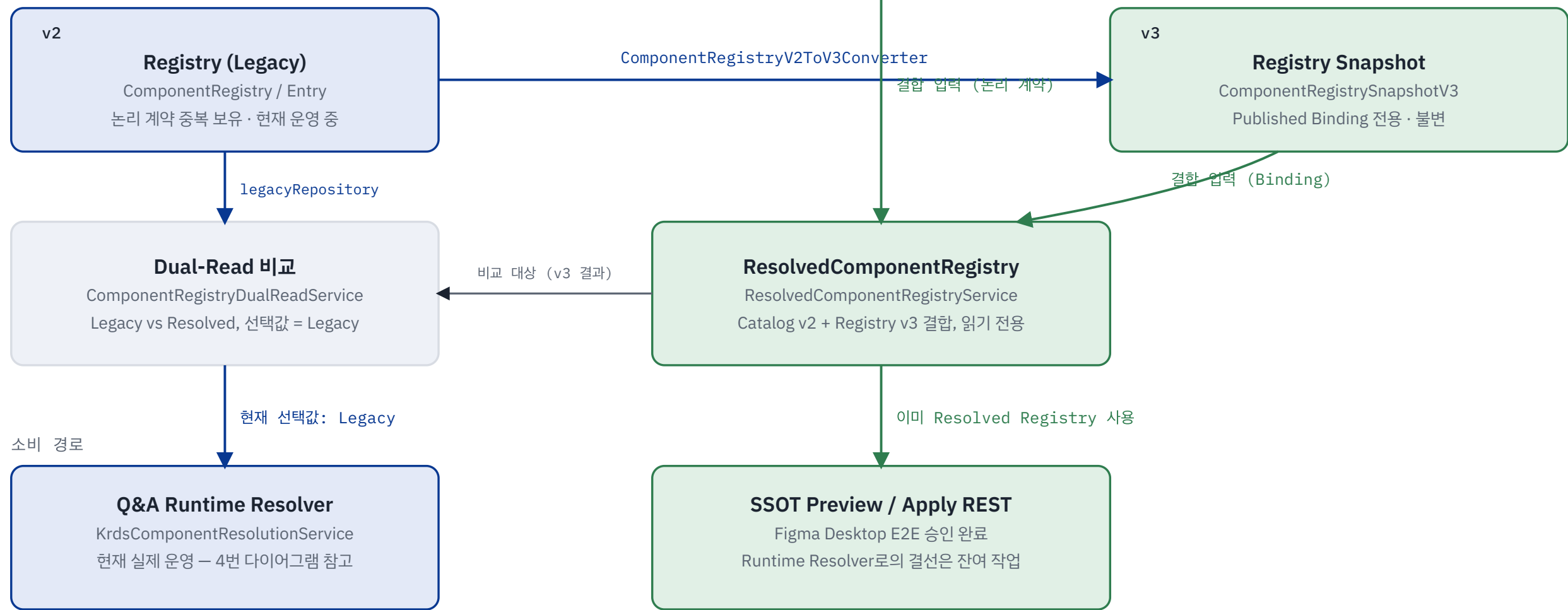
3. Component Catalog / Registry 계약 진화 — v1 → v2 → v3

"무엇을 어떻게 그릴지"를 정의하는 디자인 시스템 계약 자체가 겪은 SSOT(단일 진실 공급원) 마이그레이션입니다. Catalog와 Registry가 각자 다른 속도로 버전업하며 v1 → v2 → v3로 이어집니다(18~20번 문서).

CATALOG (논리 계약)



REGISTRY (FIGMA BINDING)

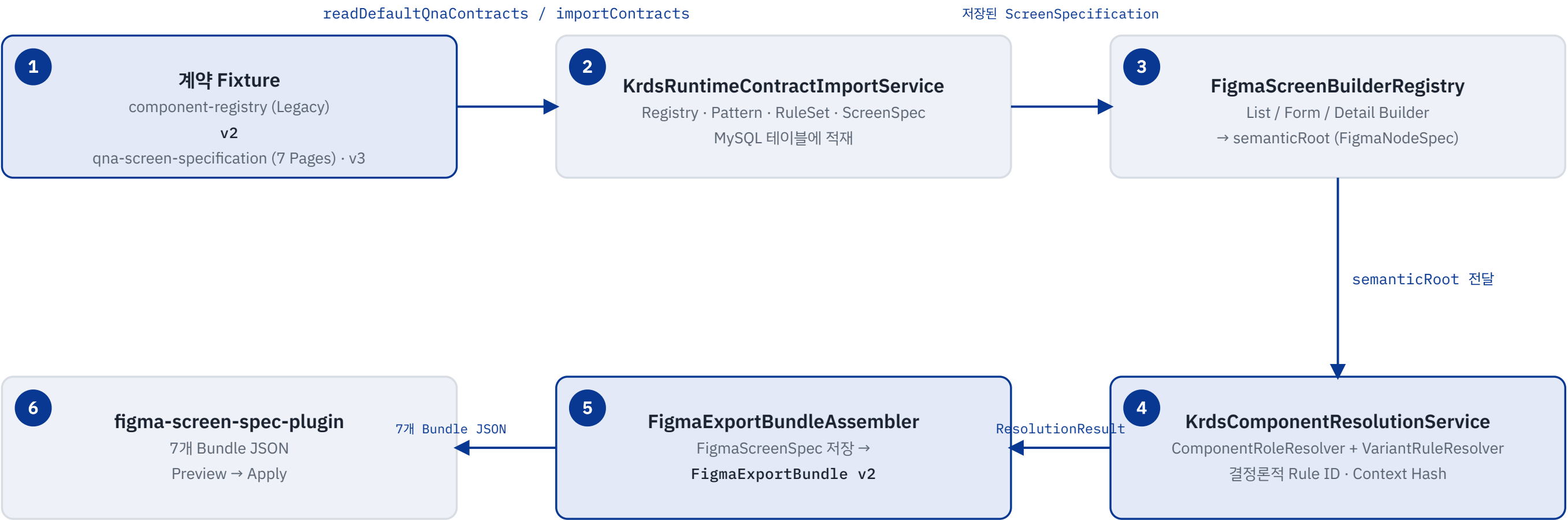


— 현재 실제 운영 경로 (Legacy v2) — 신규 SSOT 경로 (v3, 계약·REST·E2E 완료)

Catalog와 Registry는 서로 다른 것을 book-keeping합니다 — **Catalog**는 "무엇을 어떻게 그릴지"를 정하는 논리 계약(v1 → v2, 배열 기반 중복 구조를 logicalType 단일 소유 구조로 정리), **Registry**는 특정 Figma Library Publish 결과(v2 → v3, 논리 계약을 함께 들고 있던 구조를 Published Binding 전용 불변 Snapshot으로 축소)입니다. 19번 문서 (Component_Catalog_Registry_SSOT_Implementation_List.md) 기준 v2 Catalog·v3 Registry·ResolvedComponentRegistry·Preview/Apply REST·7화면 Figma Desktop E2E까지는 모두 완료됐지만, 문서 상태 문구 그대로 "기존 운영 전환은 잔여" — 실제 Q&A Runtime Resolver (KrdComponentResolutionService, import 구문 기준 여전히 Legacy ComponentRegistry 참조)는 아직 이 신규 경로로 넘어가지 않았고, ComponentRegistryDualReadService가 두 결과를 병렬 계산해 비교하는 동안 실제 선택값은 계속 Legacy입니다. MySQL 저장은 두 박스가 다릅니다 — **Registry (Legacy)**는 AI_COMPONENT_REGISTRY(v1/v2, 초안 upsert + 승인 revision-append 이중모드, 10번↔13번 다이어그램 참고)에, **Registry Snapshot**은 AI_COMPONENT_REGISTRY_V3(REST 전용, 현재 MCP Tool 경로 없음, 13번 참고)에 저장됩니다.

4. Q&A Runtime Resolver 상세 – 현재 Legacy(v2 Registry) 경로

3번 다이어그램의 "Q&A Runtime Resolver" 박스를 펼친 6단계입니다. 지금 실제로 도는 경로이므로 Registry는 Legacy v2, 반면 화면 입력(Q&A 7개 업무 화면 fixture)은 ScreenSpecification 자체 기준 v3입니다 – 3번 다이어그램의 Catalog/Registry v1~v3와는 별개의 버전 축입니다. 결정론적 재현성이 핵심이라 동일 입력을 두 번 해석해도(ResolutionResult) 같은 Rule ID·Context Hash가 나오는지 테스트로 강제합니다(KrdsQnaRuntimeResolverIntegrationTest).

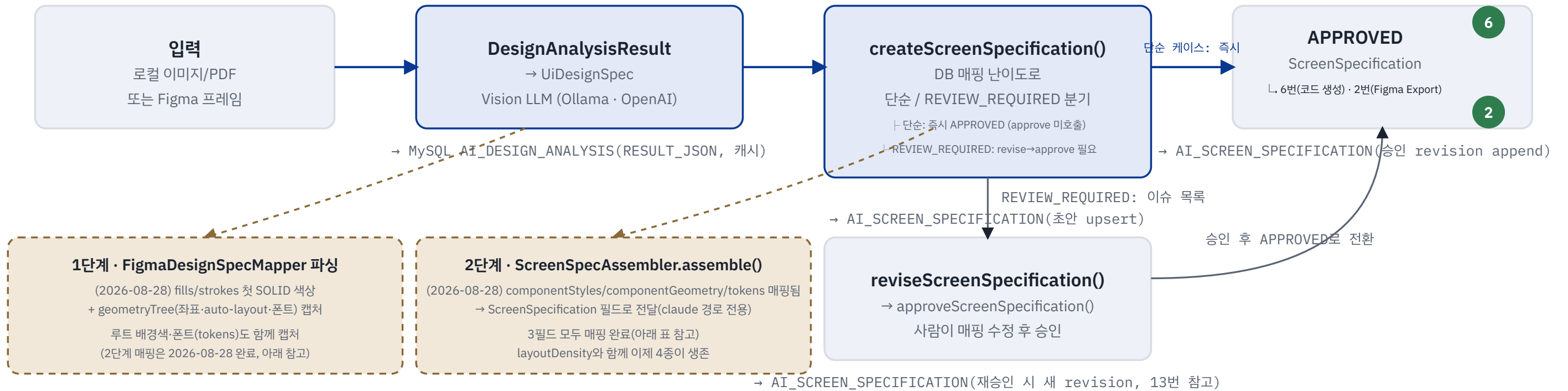


Legacy 계약 입력 → 결정론적 해석 → v2 출력

qnaRuntimeResolverTest(1~5단계)와 figmaRuntimeBundlePluginTest(6단계)가 각각 이 경계를 검증합니다. 3번 다이어그램의 ResolvedComponentRegistry(v3)가 아직 1단계의 "계약 Fixture"로 결선되지 않았다는 점이 두 다이어그램을 나눠서 그린 이유입니다 – 전환이 완료되면 1단계 Registry가 Legacy v2에서 Resolved(v3)로 바뀝니다.

5. Spring MCP 서버 상세 — DesignReferenceTool

2번 다이어그램의 **5번 배지**를 펼친 내부 흐름입니다. DB 매핑 난이도에 따라 "즉시 승인" 또는 "사람 확인 필요"로 갈라집니다.



— 정상 진행 경로 — 파싱·매핑 상세 단계 — (2026-08-28) componentStyles/componentGeometry/tokens 모두 claude 경로까지 생존, auto 경로(CrudModelFactory)만 셋 다 미참조

표준 단일 테이블처럼 JOIN·공통코드·미매핑 필드가 없는 **단순 케이스**는 buildFullCrudPrompt 호출 안에서 화면명세가 즉시 APPROVED로 생성돼 그대로 진행됩니다. JOIN·공통코드·파일 처리 등 애매한 **REVIEW_REQUIRED 케이스**만 사람이 reviseScreenSpecification으로 매핑을 고치고 approveScreenSpecification으로 승인하는 단계를 거칩니다. APPROVED가 아닌 screenSpecificationId가 전달되면 이후 생성 Tool이 예외를 던지며 파일 저장 없이 중단됩니다. 같은 APPROVED ScreenSpecification은 소비하는 Tool에 따라 **두 갈래로 갈립니다** — CrudModelFactory가 받으면 **6번 (Thymeleaf 코드)**이 되고, FigmaExportTool.generateFigmaScreenSpec()이 받으면 **2번의 Figma Export 경로** (.figma-export-bundle.json → figma-screen-spec-plugin)가 됩니다. 두 경로는 서로 독립적이며 하나가 다른 하나를 전제하지 않습니다. 이때 UiDesignSpec은 시각 구조·Token·Component 선택만 공급합니다. route, field source, validation, request/model/form binding, CSRF와 권한은 Controller·VO·DB Schema에서 수집한 업무 계약이 기준이며, 두 계약을 결합한 결과가 APPROVED ScreenSpecification입니다. 디자인 입력은 이 업무 계약을 덮어쓸 수 없습니다.

2026-08-27 갱신 — "Token·Component 공급"의 실제 폭은 훨씬 좁다. 위 문단의 "시각 구조·Token·Component 선택만 공급"이라는 표현이 마치 색상 팔레트나 컴포넌트 스타일이 어느 정도 전달되는 것처럼 읽힐 수 있어, 실제로 어느 단계에서 무엇이 살아남는지 코드로 재확인했다. 색상·컴포넌트 시각 스타일은 **두 단계**를 거치며 사실상 전부 사라진다.

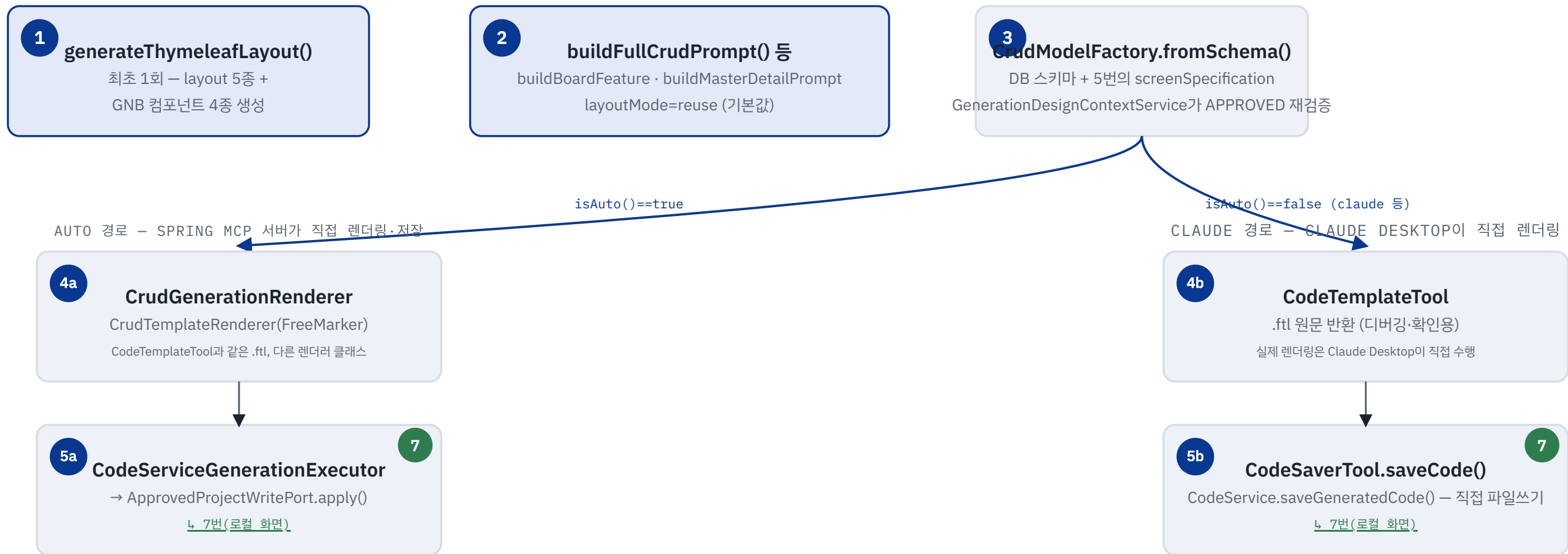
단계	대상	결과	근거
1단계 · Figma 파싱 FigmaDesignSpecMapper	컴포넌트별 fills / strokes (개별 색상)	(2026-08-28) 타입별 첫 SOLID 색상을 rgba() 로 캡처	solidFillColor() / solidStrokeColor() , 커밋 bbc8079
1단계 · Figma 파싱	노드 트리 좌표·auto-layout·텍스트스타일	(2026-08-28) geometryTree (부모-자식 구조 보존)로 캡처	buildGeometryTree() , 커밋 369cbe1
1단계 · Figma 파싱	루트 프레임 배경색·폰트	UiDesignSpec.tokens (Map<String,String>)에 캡처됨	FigmaDesignSpecMapper.java tokens()
2단계 · 매핑 ScreenSpecAssembler.assemble()	resolvedUi.components() , resolvedUi.geometryTree()	(2026-08-28) componentStyles / componentGeometry 로 매핑됨	같은 커밋 2건 — 두 getter 호출 존재
2단계 · 매핑	resolvedUi.tokens()	(2026-08-28) tokens 로 매핑됨	ScreenSpecAssembler.assemble() 마지막 인자, 커밋 4a01814
2단계 · 매핑	resolvedUi.layout().density() 등 구조값	매핑 라인이 실제로 존재 — componentStyles / componentGeometry 와 함께 생존	같은 파일, 같은 메서드
3단계 · 소비 CrudModelFactory (auto) vs ScreenSpecificationPromptFormatter (claude)	componentStyles / componentGeometry / tokens	claude 경로 프롬프트(JSON 블록 + KRDS/반응형 가드레일 + tokens 블록)에만 반영 — auto 경로는 여전히 미참조	Figma_픽셀재현_claude경로_구현계획.md §5.4, ScreenSpecificationPromptFormatter.format() tokens 블록(커밋 a1c1e30) , CrudModelFactory 참조 0건

(2026-08-28 갱신) 위 분석 이후 Figma_fills_strokes_구현계획.md(커밋 bbc8079)와 Figma_픽셀재현_claude경로_구현계획.md(커밋 369cbe1)로 이 소실 문제 일부를 해소했다. UiDesignSpec.ComponentSpec에 backgroundColor/borderColor가, UiDesignSpec에 geometryTree(부모-자식 구조·좌표·auto-layout·텍스트스타일)가 추가됐고, 둘 다 ScreenSpecification.componentStyles/componentGeometry로 매핑까지 이어진다. 루트 프레임 배경색·폰트(tokens)도 (2026-08-28) ScreenSpecAssembler.assemble() 매핑(커밋 4a01814)과 ScreenSpecificationPromptFormatter 프롬프트 반영(커밋 a1c1e30)이 마저 완료돼, 이제 componentStyles/componentGeometry와 동일하게 3단계까지 전부 생존한다. 다만 **완전히 해소된 것은 아니다** — ① componentStyles/componentGeometry/tokens 셋 다 **claude 경로(ScreenSpecificationPromptFormatter)에만** 반영되며 auto 경로 (CrudModelFactory)는 이 필드들을 전혀 참조하지 않는다(정부 표준 KRDS 룩앤필을 모든 프로젝트에서 동일하게 유지하려는 의도적 설계로 보이며, Figma_색상_CSS자동주입_ auto경로_검토.md에서 auto 경로 확장은 기각됐다). ② fills/strokes는 타입 (TABLE/SEARCH_PANEL/FORM/PAGINATION/ ACTION_GROUP)당 첫 매치 노드의 SOLID 색상 1개만 대표로 채택하고 그라데이션·이미지 채우기는 여전히 무시한다.

(2026-08-28 신규) 픽셀재현 제외범위 후속 Tool 3종. Figma_픽셀재현_제외범위_구현계획.md의 4개 트랙 중 3개가 신규 MCP Tool로 이어졌다 — compareDesignFidelity(originalAnalysisId, renderedAnalysisId)는 이 5번 섹션이 만든 원본 UiDesignSpec과, 생성된 화면을 captureWebPage() + analyzeCapturedDesign()으로 재캡처해 만든 UiDesignSpec을 비교해 archetype·컴포넌트 타입·필드 역할·액션 타입 집합의 Jaccard 유사도(구조적 일치도, 픽셀 이미지 비교 아님)를 계산한다 — 이 프로젝트에 실제 스크린샷 캡처 기능이 없다는 걸 코드로 확인하고 대신 두 경로가 이미 같은 타입을 만들어낸다는 점을 활용한 설계다. downloadFigmaAssets(analysisId, outputPath)는 FigmaDesignSpecMapper가 수집한 imageNodeIds(VECTOR 타입 또는 fills에 IMAGE 페인트가 있는 노드)를 queryImages()로 내려받아 생성 프로젝트의 static/images/figma/에 저장한다. **허용 호스트(*.figma.com)는 이 Tool을 만든 세션에 FIGMA_ACCESS_TOKEN 연동 환경이 없어 실제 API 응답으로 검증하지 못한 추정치**이며, 실사용 전 재확인이 필요하다(Figma_픽셀재현_제외범위_구현계획.md §B.5). checkResponsiveRegression(CaptureWebPageRequest)(Figma_픽셀재현_2차구현_ 반응형검증_구현계획.md, 커밋 7d543e9)는 트랙 C의 잔여 리스크 — claude 경로 프롬프트에 넣은 반응형 가드레일 문구를 실제로 지켰는지 확인할 자동화 수단이 없던 문제 — 를 해소한다. 신규 캡처·매칭 로직 없이 captureWebPageMultiViewport()가 이미 계산해두는 breakpointObservations(Desktop/Tablet/Mobile 사이 MOVED/HIDDEN/SHOWN)를 ResponsiveRegressionAnalyzer가 재분류만 해 MOVED 발생 건을 "가드레일 위반 의심" 신호로 요약한다. MOVED가 실제 위반인지 의도된 반응형 재배치인지는 구분하지 못하므로 최종 판단은 여전히 사람 몫이다.

6. Thymeleaf 생성 서비스 상세

2번 다이어그램의 **6번 배지**를 펼친 내부 흐름입니다. layout은 프로젝트당 한 번만 만들고 재사용합니다.



generateThymeleafLayout()은 프로젝트당 **최초 1회**만 실행합니다. 이후 화면 생성 Tool은 layoutMode=reuse 기본값으로 이미 만들어진 layout을 재사용 하며, layout 없이 먼저 호출하면 generateThymeleafLayout() 선행 실행을 안내하는 메시지가 반환됩니다. 5번에서 만든 screenSpecificationId는 CrudGenerationPlanner(코드 확인: generationDesignContextService.resolve(...) → crudModelFactory.fromSchema(...))를 통해 3단계로 직접 전달됩니다 - 승인 시점(5번)과 이 3단계 진입 시점 두 번 APPROVED 상태를 검증하는 이중 게이트입니다. **2026-08-26 정정:** 3단계 이후는 CrudGenerationDispatchService.isAuto()로 갈라지는 두 개의 서로 다른 경로입니다 - 이전 버전은 이 갈림을 그리지 않고 llmProvider=claude 전용 Tool(CodeTemplateTool·CodeSaverTool)을 "auto 경로 이후"인 것처럼 표시하는 오류가 있었습니다. **auto** 경로는 CrudGenerationRenderer(내부 CrudTemplateRenderer, CodeTemplateTool과 같은 .ftl 템플릿을 쓰지만 별개 클래스)가 **Spring MCP 서버**에서 직접 렌더링해 CRUD 소스 파일 (VO/Controller/ Service/Mapper/View)을 만든 뒤 CodeServiceGenerationExecutor가 ApprovedProjectWritePort로 저장합니다(14번 다이어그램의 그 Port와 동일). **claude** 경로는 CodeTemplateTool이 .ftl 원문만 반환하고(디버깅·확인 용도라고 코드 주석에 명시됨), 실제 렌더링은 Claude Desktop이 수행한 뒤 CodeSaverTool.saveCode()가 ApprovedProjectWritePort 없이 별도의 경로 검증(CodeService.validateOutputRoot)만 거쳐 직접 파일에 씁니다. 목표 상태에서는 두 경로 모두 승인된 계약으로 먼저 Thymeleaf Skeleton을 만들고 th:object, th:field, route, validation, error, CSRF를 적용한 뒤, Thymeleaf parse/render와 binding audit, 프로젝트 build, Desktop·Tablet·Mobile 렌더 검증을 모두 통과해야 완료됩니다. 즉 렌더링과 파일 저장은 중 점이 아니라 검증 게이트의 입력입니다.

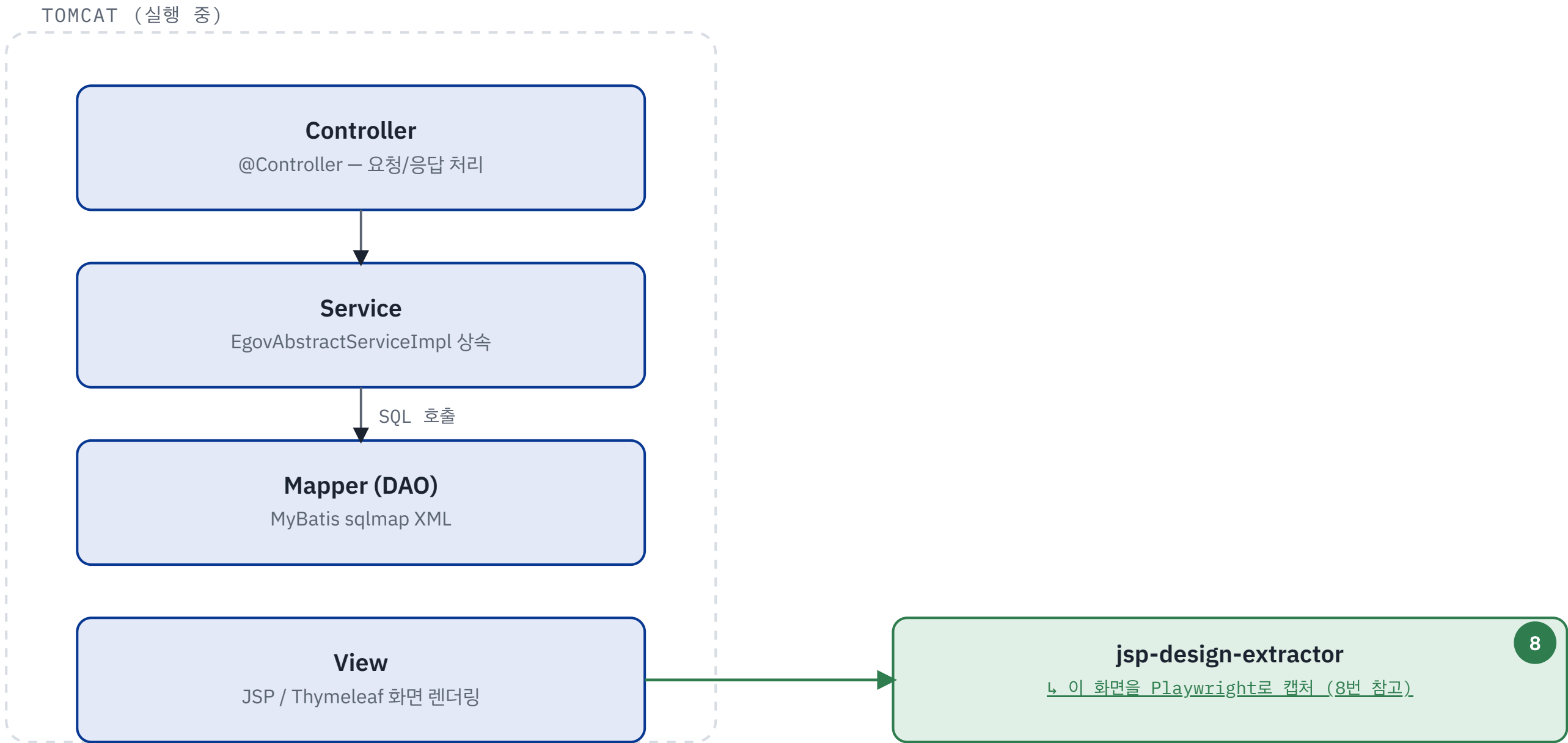
2026-08-27 갱신 - 3번 박스(CrudModelFactory.fromSchema())는 사실 auto 전용이다. 위 다이어그램은 3번 박스를 isAuto() 분기 이전 공통 단계처럼 그리고 있지만, 실제 분기(CrudGenerationDispatchService.isAuto())는 CrudModelFactory가 호출되기도 전, Dispatch 최상단에서 일어난다. **auto** 경로만 CrudGenerationPlanner → CrudModelFactory.fromSchema()를 타고, **claude** 경로는 이 클래스를 아예 거치지 않고 완전히 별개인 CrudPromptGenerationService → CrudPromptBuilderService .buildFullCrudPrompt()로 간다. 두 경로가 5번의 ScreenSpecification을 소비하는 방식도 범위가 다르다 - CrudModelFactory(service/CrudModelFactory.java L107-216)는 selectionSource 기반 필드 선택과 layoutDensity/ formColumnLayout/actionPlacement/searchPanelPlacement 4개 구조값만 읽어 FreeMarker 변수로 넘기고(archetype·dataSources/조인·필드 라벨은 이 클래스에서 아예 안 읽음), CrudPromptBuilderService(L206)는

ScreenSpecificationPromptFormatter(L12-53)를 통해 archetype·전체 dataSources(조인 포함)·페이지별 필드 label/dataRole/source/actions까지 통째로 텍스트로 프롬프트에 삽입한다 — auto보다 훨씬 넓은 범위를 전달하지만 구조화된 값이 아니라 텍스트 설명이라, 실제로 코드에 반영할 지는 그 프롬프트를 읽는 Claude의 해석에 달려 있다.

항목	AUTO — CRUDMODELFACTORY	CLAUDE — CRUDPROMPTBUILDERSERVICE
ScreenSpecification 반영 범위	표 밀도 4종 + selectionSource 기반 필드 선택	위 전부 + archetype + dataSources /조인 + 필드 라벨·역할·액션
전달 형태	구조화된 값 → FreeMarker 템플릿 변수	통째로 텍스트 설명 → 프롬프트
표 밀도 CSS 반영	CrudTableDensityCssProcessor가 styles.css에 실제 CSS 자동 주입 (TableDensityCssContract)	없음 — "밀도는 X"라고 텍스트로만 전달, CSS는 Claude가 직접 작성해야 함
Figma 색상·좌표·토큰 (componentStyles / componentGeometry / tokens)	반영 안 됨 — CrudModelFactory가 이 필드들을 참조하지 않음(의도된 KRDS 일관성 유지, 5번 섹션 참고)	(2026-08-28) ScreenSpecificationPromptFormatter가 JSON 블록(색상·좌표) + tokens 블록 + KRDS/반응형 가이드라인 문구로 프롬프트에 포함

7. 로컬 화면 상세 – eGovFrame 4계층

2번 다이어그램의 7번 배지를 펼친 것입니다. 이 박스는 springai가 만들어낸 코드가 실행되는 "결과물"이라, 흐름이 아니라 계층 구조를 보여줍니다.



이 4계층은 springai 저장소 안에 있지 않습니다 – springai가 만들어낸 **대상 eGovFrame 프로젝트**에서 Tomcat이 직접 실행하는 코드입니다. Controller가 Service를, Service가 Mapper를 호출하는 표준 eGovFrame 레이어드 아키텍처를 그대로 따릅니다. **2026-08-26 정정:** 파일을 실제로 쓰는 주체는 tools/CodeSaverTool 하나가 아니라 6번 다이어그램에서 정리한 두 경로 중 하나입니다 – llmProvider=auto면 CodeServiceGenerationExecutor가 ApprovedProjectWritePort로, claude 등이면 tools/CodeSaverTool.saveCode()가 CodeService.saveGeneratedCode()로 저장합니다. 어느 경로로 쓰였든 결과 파일과 여기서 그리는 실행 계층 구조는 동일합니다.

8. jsp-design-extractor 상세

2번 다이어그램의 8번 배지를 펼친 내부 흐름입니다(05_Overall_Architecture_Diagram.md의 E1~E4 단계와 대응).

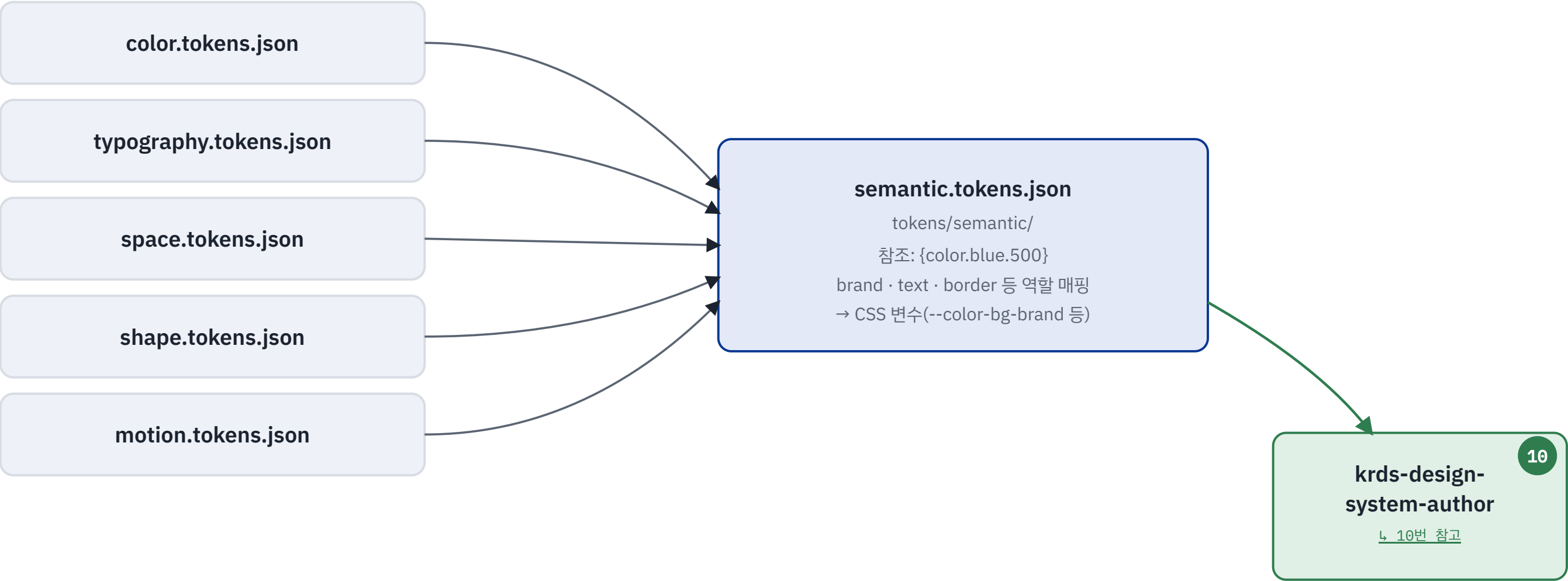


`npm run test:browser-gate`가 완성된 HTML 문자열 또는 loopback URL을 입력받아 세 viewport 전부에서 수평 overflow, JS/리소스 오류, axe 접근성, PNG visual diff를 검증합니다. 산출물 `figpack-v1`은 `website-figma-contract`의 계약이라 2번 다이어그램의 `jsp-to-figma-plugin`이 그대로 소비할 수 있습니다.

9. component-contracts 상세 — 디자인 토큰 구조

2번 다이어그램의 9번 배지를 펼친 내부 구조입니다. W3C Design Tokens 형식(\$value/\$type)을 따르며, Primitive를 Semantic이 참조합니다.

TOKENS/PRIMITIVE/



Primitive는 {"\$type": "color"}처럼 원시값만 갖고, Semantic이 {color.blue.500} 참조 문법으로 이를 가져와 brand·text·border 같은 역할 이름에 매핑합니다. 3번 다이어그램의 Component Catalog/Registry(컴포넌트 논리 계약)와는 완전히 다른 축 — 이쪽은 색상·간격 같은 **토큰 값** 자체를 다룹니다.

10. krds-design-system-author-plugin 상세

2번 다이어그램의 10번 배지를 펼친 내부 흐름입니다. 마지막 단계는 3번 다이어그램의 "Registry (Legacy)"로 이어집니다.

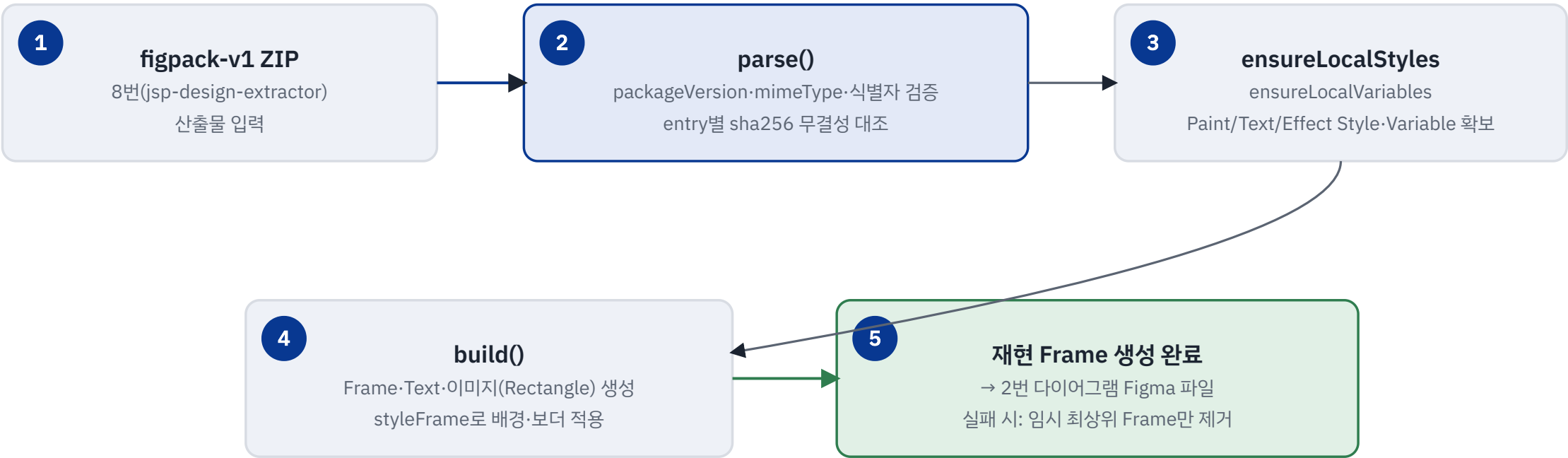


→ MySQL AI_COMPONENT_REGISTRY · AI_DESIGN_SYSTEM_PROFILE · AI_FIGMA_REVIEW_HISTORY 기록(13번)

Figma Plugin API는 Team Library Publish를 프로그래밍적으로 수행할 수 없어 5단계는 반드시 사람이 Figma UI에서 직접 합니다. Plugin은 현재 파일의 fileKey를 제공하지 않으므로 Registry 내보내기 화면에서 Library fileKey를 직접 입력해야 합니다(FTC 테스트 Library: mVy5h1Ub0RVqQoBm8Wr1bT). ComponentRegistrySyncService는 모든 필수 자산이 CURRENT일 때만 사람 확인 후 불변 Registry Snapshot을 저장하고 Profile을 PUBLISHED로 전환합니다 — 이게 바로 3번 다이어그램 "Registry (Legacy)" 박스의 실제 입력 출처입니다.

11. jsp-to-figma-plugin 상세

2번 다이어그램의 **11번 배지**(및 8번 다이어그램 출력)를 펼친 내부 흐름입니다. 네트워크에 접근하지 않는 로컬 전용 Plugin이며 실패 시 이번 실행분만 롤백합니다.



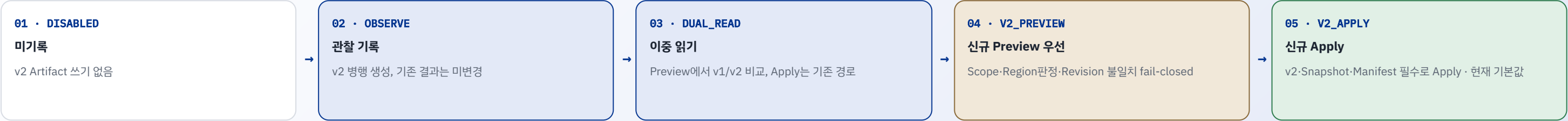
2단계 parse()가 manifest.json/document.json의 식별자 (captureId·documentKey·contentHash) 일치와 entry별 sha256 hash를 전부 대조해 위·변조나 손상된 figpack-v1을 초기에 차단합니다. Plugin은 네트워크에 접근하지 않고, 4단계 build() 도중 실패하면 이번 실행에서 만든 임시 최상위 Frame("IMPORTING {title}")만 제거해 Figma 파일의 다른 내용을 건드리지 않습니다.

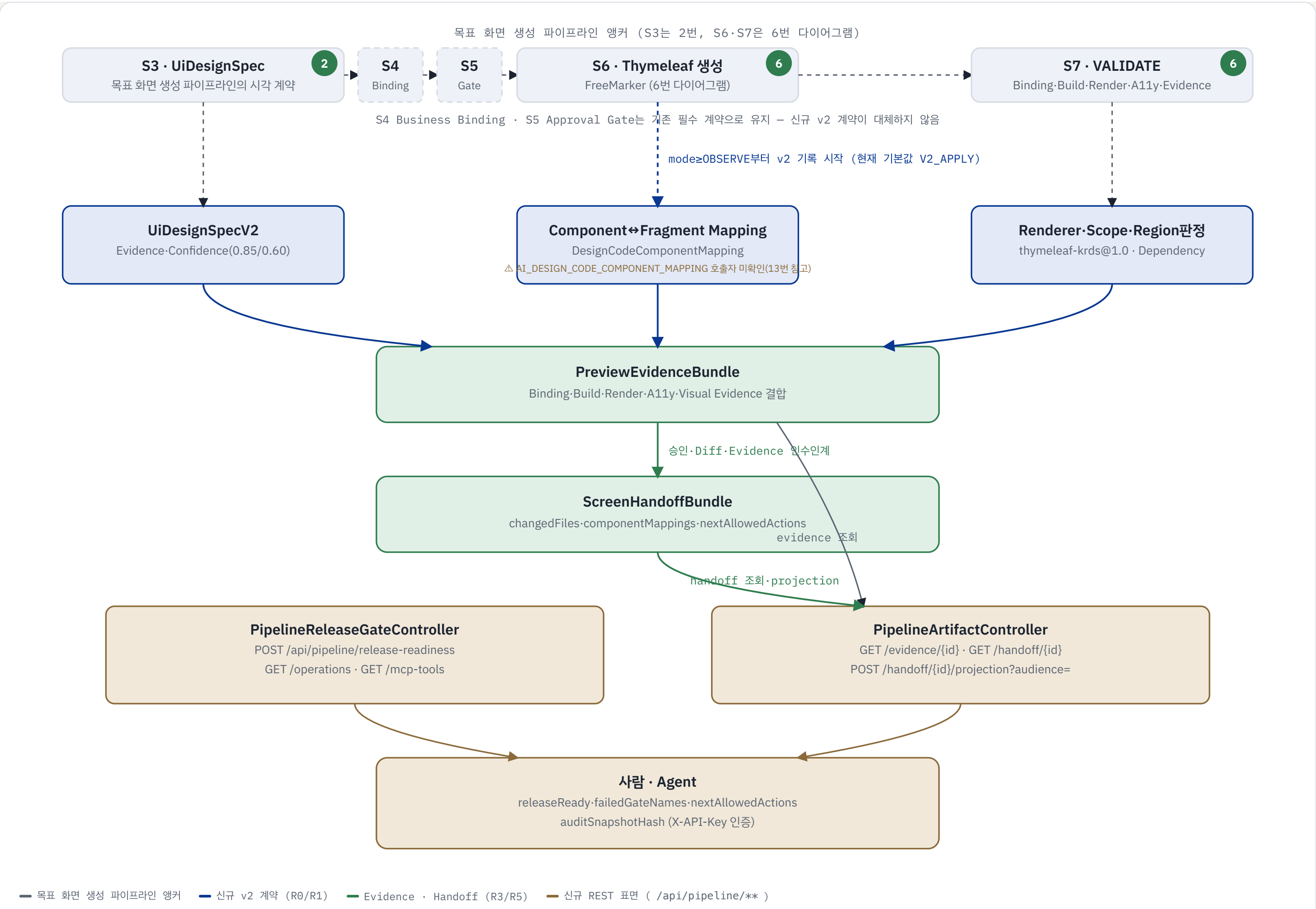
12. Pipeline Evolution — 목표 화면 생성 파이프라인의 Release Gate와 신규 계약 신규 반영

5축 벤치마크 명세(29·30번 문서)의 R0~R5 계약이 feat: complete 5-axis pipeline release gates 커밋으로 실제 코드에 반영되었습니다. 이 계층은 **목표 화면 생성 파이프라인**(2번 다이어그램 2.1절에서 정의한 그 S1~S7 파이프라인 — 이 문서의 유일한 S-번호 파이프라인입니다)의 S3·S6·S7단계 위에 얹히는 **확장**입니다(S3는 2번 다이어그램에, S6·S7은 6번 다이어그램에 표시됩니다. S4 Business Binding·S5 Approval Gate는 기존 계약 그대로 유지). app.pipeline-evolution.mode의 기본값은 8445977 feat: default pipeline-evolution.mode to V2_APPLY 커밋 이후 V2_APPLY로 전환되어(application.yaml:122), 운영 auto 생성 경로에 이미 적용되는 상태입니다. 장애·롤백 시에는 APP_PIPELINE_EVOLUTION_MODE 환경변수로 V2_PREVIEW 이하 단계로 단계적으로 낮출 수 있습니다.

PipelineEvolutionProperties.mode 전환 단계

app.pipeline-evolution.mode 하나의 Flag가 v2 계약의 쓰기·읽기·Preview·Apply 사용 여부를 전부 통제합니다. autoApprovalConfidenceThreshold=0.85, evidenceConfidenceThreshold=0.60 는 모든 Mode에서 공통으로 적용되는 시각 추론 자동 승인 임계값입니다.

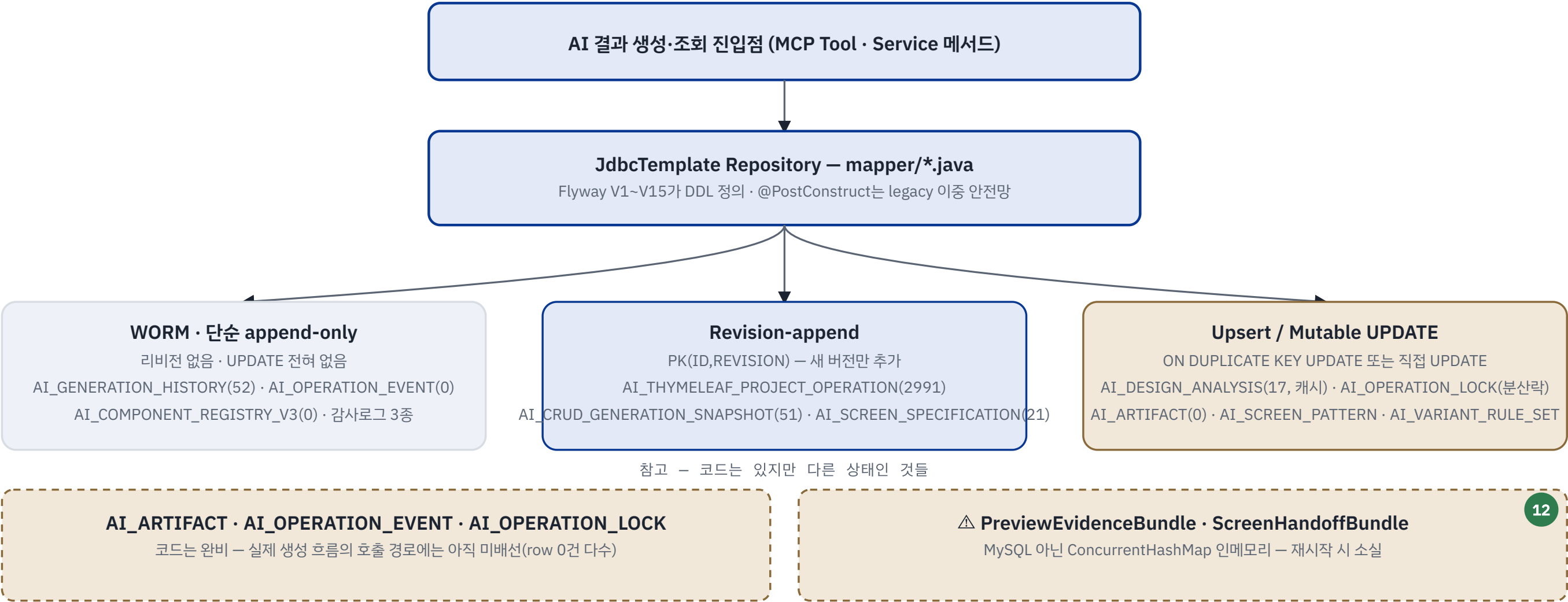




commit d009058 "feat: complete 5-axis pipeline release gates"가 이 계층 전체를 한 번에 추가했습니다(299 files changed, +22394). S4 Business Binding(DB·Controller·VO·권한)과 S5 Approval Gate (ScreenSpecification.status == APPROVED)는 기존 계약 그대로 유지되며 이번 확장의 대상이 아닙니다. S3단계는 UiDesignSpecV2(Source·Evidence·Confidence·Renderability), S6단계는 DesignCodeComponentMapping(Figma Property → Fragment Parameter)과 RendererProfile·GenerationScopeManifest·GenerationOwnershipManifest로 확장됩니다. S7단계 이후 결과는 PreviewEvidenceBundle → ScreenHandoffBundle 순으로 조립되고, 이 두 Bundle은 **기존 /api/**와 별개인 신규 /api/pipeline/**** 표면으로 노출됩니다: PipelineReleaseGateController는 호출자가 전달한 Binding·Build·Render·Accessibility Gate 결과를 PipelineReleaseReadiness로 평가해 ready/failedGateNames를 돌려주고, PipelineArtifactController는 저장된 Evidence·Handoff Bundle을 조회하거나 audience(DSIGNER·BUSINESS·DEVELOPER·QA·APPROVER·AGENT)별 Projection을 releaseReady·nextAllowedActions·auditSnapshotHash와 함께 반환합니다. 1번 다이어그램과 동일하게 이 신규 REST 표면도 X-API-Key만으로 호출되며 MCP Tool 위험 등급 인가 게이트를 거치지 않습니다. **기본값이 V2_APPLY**로 전환되어 이 도표의 흐름이 이미 운영 auto 생성 경로에 적용되는 상태입니다 — 자세한 배포 전 점검 절차는 docs/figma/31_5Axis_Pipeline_Release_Gate_Operations_Runbook.md를 참고하십시오.

13. MySQL AI_* 테이블 데이터 흐름 — 생성·갱신 계약 신규 반영

1번 다이어그램의 **MySQL** 박스 하나를 펼친 것입니다. Flyway 마이그레이션(src/main/resources/db/migration/V1~V15__*.sql, spring.flyway.baseline-version=1)이 정의하는 AI_* 테이블 20종을 실제 운영 DB(docker exec egov-mysql mysql -u ebt -pebt01 ebt -e "SHOW TABLES")에 대조해 코드-DB 드리프트가 없음을 확인했습니다. 다수 Repository는 @PostConstruct의 CREATE TABLE IF NOT EXISTS를 병행하지만, V1 마이그레이션 주석 기준으로 이는 "9개 legacy Repository의 이중 안전망"일 뿐이며 **DDL의 진실은 Flyway**입니다.



— WORM · append-only — Revision-append (불변 버전) — Upsert · Mutable UPDATE / 참고 (미배선·인메모리)

20개 AI_* 테이블은 세 가지 영속 계약 중 하나를 따릅니다. **WORM(append-only)**은 INSERT만 있고 UPDATE가 전혀 없는 불변 로그(예: AI_GENERATION_HISTORY). **Revision-append**는 PK에 REVISION 컬럼을 포함해 수정 대신 항상 MAX(REVISION)+1 새 행을 추가하는 방식(예: AI_THYMELEAF_PROJECT_OPERATION, AI_CRUD_GENERATION_SNAPSHOT) — 승인·재현성이 중요한 산출물에 씁니다. **Upsert/Mutable UPDATE**는 ON DUPLICATE KEY UPDATE 또는 직접 UPDATE로 값을 덮어씁니다(예: AI_DESIGN_ANALYSIS는 SOURCE_HASH+PROVIDER_ID+MODEL_ID+PROMPT_VERSION 기준 캐시, AI_OPERATION_LOCK은 분산락). AI_SCREEN_SPECIFICATION과 AI_COMPONENT_REGISTRY(v1/v2)는 초안 단계에서는 save()로 upsert하다가 승인 시점에 saveImmutable()로 전환되는 **이중 모드**입니다. AI_ARTIFACT·AI_OPERATION_EVENT·AI_OPERATION_LOCK은 Repository 코드가 이미 존재하지만 실측 row가 0건 — 설계는 끝났으나 실제 CRUD 소스 생성 흐름의 호출부에서 아직 이 테이블들을 호출하지 않는 **미배선** 상태입니다. 테이블별 정확한 파일·라인 근거는 아래 표를 참고하십시오.

⚠ 발견: 12번 다이어그램의 Evidence·Handoff Bundle은 MySQL이 아니라 인메모리입니다

PreviewEvidenceBundle·ScreenHandoffBundle·GenerationJob·ScreenReviewSession·DesignSystemKnowledgeSnapshot – 12번 다이어그램(Pipeline Evolution Release Gate)이 그리는 Row 3·4의 산출물은 전부 ConcurrentHashMap 기반 인메모리 Repository입니다(service/evidence/PreviewEvidenceBundleRepository.java:14, service/handoff/ScreenHandoffBundleRepository.java:5, service/job/GenerationJobRepository.java:5, service/review/ScreenReviewSessionRepository.java:11). MySQL 테이블은 전혀 없어 **애플리케이션 재시작 시 전부 소실**됩니다.

같은 도표의 AI_THYMELEAF_PROJECT_OPERATION은 V2 마이그레이션 주석에서 정확히 이런 "재시작 시 승인 작업 소실" 문제를 MySQL로 이관해 해결한 선례입니다. Evidence·Handoff Bundle은 아직 그 이관이 이뤄지지 않은 상태이며, 12번 다이어그램의 app.pipeline-evolution.mode 기본값이 이미 V2_APPLY로 운영 경로에 적용 중이라는 점(위 12번 섹션 참고)과 맞물려 재점검이 필요한 지점입니다.

테이블	패턴	핵심 컬럼(AI 결과물)	생성 주체	갱신 방식	실측 ROW
AI_GENERATION_HISTORY	WORM	TABLE_NAME, DOMAIN, PACKAGE_NAME, OUTPUT_PATH, GENERATED_FILES	GenerationHistoryRepository.insert() mapper/GenerationHistoryRepository.java:52	없음(append-only)	52
AI_DESIGN_ANALYSIS	Upsert(캐시)	RESULT_JSON(LONGTEXT), SOURCE_HASH+PROVIDER_ID+MODEL_ID+PROMPT_VERSION UNIQUE	DesignAnalysisRepository.saveOrUpdate() mapper/DesignAnalysisRepository.java:53	INSERT ... ON DUPLICATE KEY UPDATE(사실상 캐시 재사용)	17
AI_SCREEN_SPECIFICATION	Revision + Upsert(이중모드)	SPEC_STATUS, SPEC_JSON, PK(SPEC_ID, SPEC_VERSION)	ScreenSpecRepository.save() / saveImmutable() mapper/ScreenSpecRepository.java:51-79	초안=upsert, 승인본=새 버전 append	21
AI_COMPONENT_REGISTRY (v1/v2)	Revision + Upsert(이중모드)	REGISTRY_JSON, PK(PROFILE_ID, REGISTRY_VERSION)	ComponentRegistryRepository.save() / saveImmutable() mapper/ComponentRegistryRepository.java:48-62	save()=upsert, saveImmutable()=중복 시 예외	4
AI_COMPONENT_REGISTRY_V3	WORM	CATALOG_VERSION, SCHEMA_VERSION, SOURCE_REVISION, APPROVED_BY/AT, CONTENT_HASH, REGISTRY_JSON	ComponentRegistrySnapshotV3Repository.saveImmutable() mapper/ComponentRegistrySnapshotV3Repository.java:23-32	없음(WORM)	0(미사용)
AI_DESIGN_CODE_COMPONENT_MAPPING	WORM	MAPPING_STATUS, LOGICAL_TYPE, FIGMA_COMPONENT_SET_KEY, THYMELEAF_FRAGMENT, APPROVED_BY/AT, CONTENT_HASH, MAPPING_JSON	DesignCodeComponentMappingRepository.saveImmutable() mapper/DesignCodeComponentMappingRepository.java:23-34	없음(WORM)	미확인(신규)
AI_FIGMA_DESIGN_OPERATION + _IDEMPOTENCY	Revision	STATUS(ANALYZED/CONFLICT/APPLIED 등), OPERATION_JSON, PK(OPERATION_ID, REVISION)	FigmaDesignOperationRepository.createOrReuse() mapper/FigmaDesignOperationRepository.java:93-128	appendTransition() / transitionToApplied() 가 다음 revision INSERT(:135-170)	3
AI_FIGMA_GENERATION_REPORT	WORM	SUCCESS_YN, REPORT_JSON	FigmaGenerationReportRepository insert	없음	미확인
AI_FIGMA_REVIEW_HISTORY	WORM	EVENT_TYPE, EVENT_STATUS, ACTOR, COMMENT_TEXT	FigmaReviewHistoryRepository insert	없음(append-only 감사로그)	미확인
AI_FIGMA_SCREEN_SPEC	Revision	SPEC_JSON, PK(SCREEN_ID, SCREEN_VERSION)	FigmaScreenSpecRepository insert	신규 버전 append	미확인
AI_FIGMA_REFINEMENT_SET / _PATCH	Upsert	REFINEMENT_STATUS, APPROVED_BY/AT / BEFORE_VALUE_JSON, AFTER_VALUE_JSON, CONFLICT_STATUS	FigmaRefinementRepository insert	UPDATE AI_FIGMA_REFINEMENT_SET SET ... (승인 상태 전이) mapper/FigmaRefinementRepository.java:154	미확인
AI_FIGMA_LIBRARY_INVENTORY	WORM	SNAPSHOT_JSON, PK(PROFILE_ID, REGISTRY_VERSION, INVENTORY_VERSION)	insert-only 스냅샷	없음	미확인

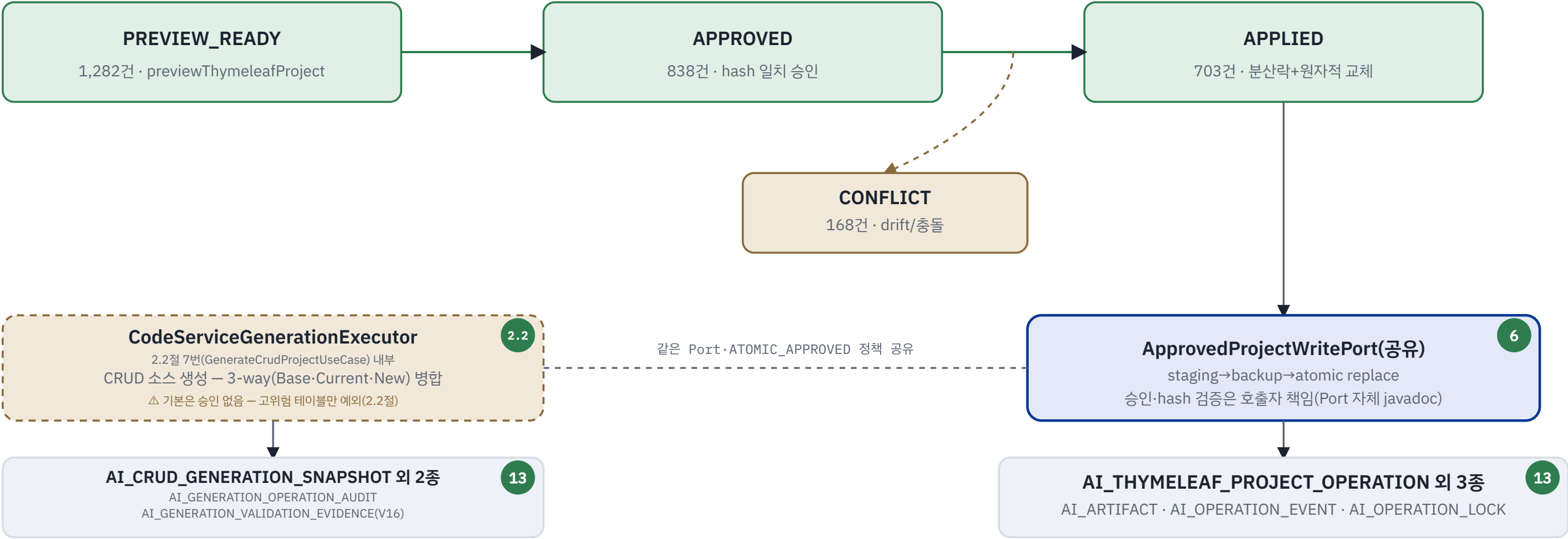
테이블	패턴	핵심 컬럼(AI 결과물)	생성 주체	갱신 방식	실측 ROW
AI_SCREEN_PATTERN	Upsert	PATTERN_STATUS, PATTERN_JSON	insert	UPDATE AI_SCREEN_PATTERN SET PATTERN_STATUS=?, PATTERN_JSON=?	미확인
AI_VARIANT_RULE_SET	Upsert	RULE_SET_STATUS, RULE_SET_JSON	insert	UPDATE ... SET RULE_SET_STATUS, RULE_SET_JSON	미확인
AI_CRUD_GENERATION_SNAPSHOT	Revision	SNAPSHOT_JSON(GenerationOwnershipManifest 전체), PK(OPERATION_ID, REVISION)	CrudGenerationSnapshotRepository. save() mapper/CrudGenerationSnapshotRepository.java:42-58	MAX(REVISION)+1 계산 후 append	51
AI_THYMELEAF_PROJECT_OPERATION + _IDEMPOTENCY + _SCREEN_OPERATION_INDEX	Revision(+색인 upsert)	STATUS, SNAPSHOT_JSON	ThymeleafProjectOperationRepository mapper/ThymeleafProjectOperationRepository.java:73-119	INDEX 테이블만 ON DUPLICATE KEY UPDATE	2991(최다 사용)
AI_GENERATION_OPERATION_AUDIT	WORM(신설 · 14.1절 참고)	OPERATION_ID, OPERATION_REVISION, CHANGED_REGIONS_JSON, PRESERVED_REGIONS_JSON, CONFLICT_REGIONS_JSON, STATUS, PK(AUDIT_ID)	GenerationControlPlaneRepository. append(GenerationAuditRecord) mapper/GenerationControlPlaneRepository.java:32-35	없음(append-only, CONFLICT·FAILED·APPLIED 매 시도)	미확인(신규)
AI_GENERATION_VALIDATION_EVIDENCE	WORM(신설 · 14.1절 참고)	OPERATION_ID, GATE_TYPE, STATUS, SEVERITY, CONTENT_HASH, PK(EVIDENCE_ID)	GenerationControlPlaneRepository. append(ValidationEvidence) mapper/GenerationControlPlaneRepository.java:65-67	없음(append-only, RENDER·BUILD·BINDING Gate마다)	미확인(신규)
AI_GENERATION_READINESS_COMPARISON	WORM(신설 · 14.1절 참고)	OPERATION_ID, SOURCE_TYPE, LEGACY_READY, COMMON_READY, MATCHED, PK(COMPARISON_ID)	ReadinessComparisonRepository. append(ReadinessComparisonRecord) mapper/ReadinessComparisonRepository.java:29-31	없음(append-only, 30일 관측용)	미확인(신규)
AI_ARTIFACT + _LINK	Upsert	ARTIFACT_TYPE, MEDIA_TYPE, CONTENT_HASH(UNIQUE), STORAGE_URI, STATUS	ArtifactCatalogRepository insert :51, 86	UPDATE AI_ARTIFACT SET STATUS=? WHERE ARTIFACT_ID=? (:105)	0(미사용/미배선)
AI_OPERATION_EVENT	WORM	FROM_STATUS, TO_STATUS, EVENT_TYPE, ACTOR, CORRELATION_ID	OperationEventRepository insert	없음(append-only 이벤트로그)	0(미사용/미배선)
AI_OPERATION_LOCK	Upsert(분산락)	OWNER_ID, FENCING_TOKEN, EXPIRES_AT	분산락 획득 시 insert	UPDATE AI_OPERATION_LOCK SET ... WHERE LOCK_KEY=? (SELECT ... FOR UPDATE 동반)	미확인
AI_DESIGN_SYSTEM_PROFILE	Upsert	PROFILE_STATUS, PROFILE_JSON, UPDATED_AT(ON UPDATE 트리거)	insert	DB 트리거로 자동 갱신	미확인
AI_COMPONENT_REGISTRY_RESOLUTION_REPO RT	WORM	IDENTICAL(BOOLEAN), REPORT_JSON	insert-only 비교 리포트	없음	미확인

미확인 표기는 Flyway/코드 정적분석으로 존재는 확인했으나 실제 row 수를 직접 조회하지 않은 항목입니다(SHOW TABLES만 전수 확인, 일부만 COUNT(*) 실측). Redis 기반 RAG 벡터 스토어·채팅 메모리(EgovRedisChatMemoryRepository, chat:memory:{conversationId} 키)는 MySQL이 아니므로 이 표에서 제외했습니다 — 1번 다이어그램의 **Redis Stack** 박스가 그 경로입니다.

14. Thymeleaf 마이그레이션 승인 워크플로우 – CRUD 소스 생성과의 승인 단계 비교 신규 반영

13번에서 최다 사용 테이블로 드러난 AI_THYMELEAF_PROJECT_OPERATION을 실제로 쓰는 주체입니다. 레거시 JSP 화면을 Thymeleaf로 변환해 덮어쓰는, 위험도가 다른 별도 승인 상태기계이며, 2번 다이어그램의 2.2절(CRUD 소스 생성 경로)과는 한 번도 서로의 상위 서비스가 되지 않지만 파일 쓰기 계층(ApprovedProjectWritePort)은 설계 시점부터 공유하도록 되어 있습니다.

실측 상태 분포 (2026-08-03~08-24, AI_THYMELEAF_PROJECT_OPERATION – REJECTED/VALIDATED/FAILED 0건)



— 정상 승인 경로 — 충돌/비대칭 지점 — 공유 인프라

ProjectOperationStatus enum은 9개 값(ANALYZED, CONTRACT_READY, PREVIEW_READY, APPROVED, APPLIED, VALIDATED, FAILED, CONFLICT, REJECTED)을 정의하지만, 실측 DB에는 4개 값만 존재합니다. ANALYZED·CONTRACT_READY는 preview() 호출 한 번 안에서 즉시 PREVIEW_READY로 전이돼 저장되는 트랜지언트 상태로 보이며(미확인), VALIDATED·FAILED·REJECTED가 0건인 것은 revalidateThymeleafProject(WithBrowserGate) 재검증 단계가 실제 운영에서 아직 호출되지 않았거나 항상 APPLIED에 머물러 있다는 뜻으로 읽힙니다(미확인). approve()는 상태가 정확히 PREVIEW_READY이고 previewHash가 일치하며 validationErrors가 비어 있을 때만 통과합니다 — 셋 중 하나라도 어긋나면 APPROVAL_REJECTED 이벤트만 남기고 예외를 던집니다. apply()는 분산락(AI_OPERATION_LOCK, 키 "THYMELEAF:" + projectRoot, 5분 타임아웃)을 먼저 잡은 뒤 DESIGN.md/legacy source drift를 재검사하고, 문제 없으면 ApprovedProjectWritePort로 원자적 교체합니다.

핵심 결론

CRUD 소스 생성과 Thymeleaf 마이그레이션은 동일한 파일 쓰기 Port를 공유하지만, 위험 모델과 승인 방식이 다르므로 실행기는 분리한다. CRUD 소스는 반복 재생성을 위해 Region 3-way(Base·Current·New) 비교 기반 자동 병합을 사용하고, Thymeleaf 마이그레이션은 레거시 화면 변환의 위험도를 고려해 Preview Hash·Source Revision·명시적 승인·Revalidate를 강제한다. 다만 2026-08-26부터 고위험으로 지정한 테이블에 한해 CRUD 소스 생성에도 조건부 승인 게이트(CrudGenerationApprovalPolicy, 2.2절)가 예외로 적용될 수 있다(기본값은 비활성).

통합 대상은 실행 알고리즘이 아니라 **Generation Operation, Validation Evidence, Approval State, Conflict·Rollback Audit**과 같은 제어 계층이며, 이 제어 계층은 **14.1절에서 실제로 구현·커밋됐다** (GenerationControlPlaneService, 커밋 5ae8507). 다만 이는 위 CodeServiceGenerationExecutor의 "명시적 승인 단계 없음" 문제 자체를 고친 것이 아니다 — 14.1절의 Approval 값은 CRUD 소스 쪽이 여전히 AUTOMATED_OWNERSHIP_CHECK(사람 승인 없음)이므로, **승인 공백이 해소된 게 아니라 그 공백이 API로 관측 가**

능해졌을 뿐이다. ATOMIC_APPROVED는 승인 방식이 아니라 원자적 파일 쓰기 정책이므로, 사람 승인과 자동 3-way 판정 승인을 별도 계약으로 표현해야 한다.

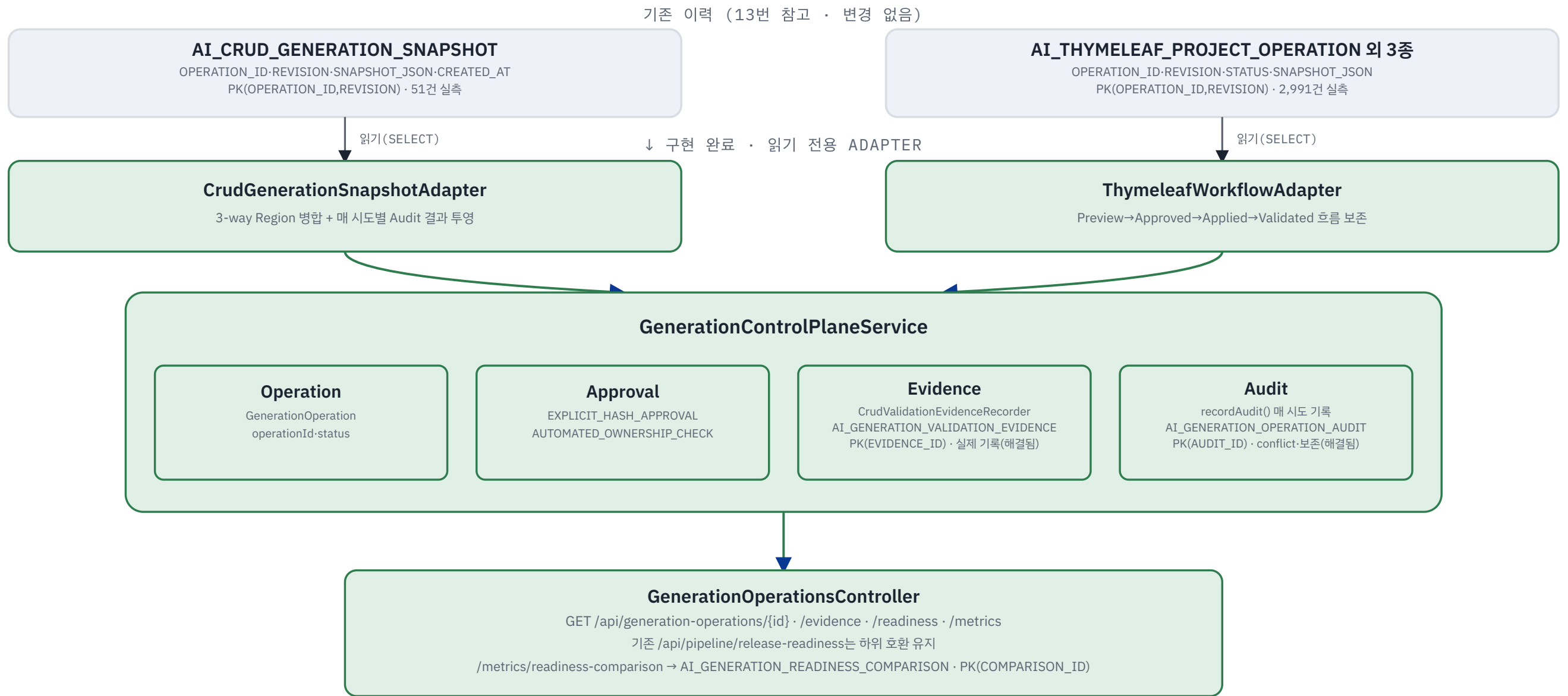
같은 이유로 위 CodeServiceGenerationExecutor 박스는 여전히 주황(rest)+점선으로 남아 있다. 이 점선이 표시하는 건 **기본 동작**이지 예외 경로가 아니기 때문이다. CrudGenerationApprovalPolicy는 기본값이 꺼져 있는(approval-required-tables가 빈 목록) 선택적 게이트이므로, 운영자가 특정 테이블을 명시적으로 등록하지 않는 한 지금 이 순간에도 CRUD 소스 생성은 승인 없이 그대로 저장된다. 바뀐 건 "예외 경로 하나가 추가됐다"는 것이지 "기본 상태에 뚫려 있던 구멍이 막혔다"는 것이 아니다 — 그래서 박스 문구도 실선 전환 없이 "△ 기본은 승인 없음 — 고위험 테이블만 예외(2.2절)"로만 갱신했다. 만약 반대로 approval-required-for-all이 기본값 true였다면, 그때는 "기본적으로 항상 승인이 걸린다"는 뜻이므로 점선을 실선으로 바꾸는 게 맞았을 것이다.

축	CRUD 소스 생성 경로(2.2절)	THYMELEAF 마이그레이션(14번)
진입점	<code>buildFullCrudPrompt</code> 등 → <code>GenerateCrudProjectUseCase</code>	<code>previewThymeleafBindingGeneration</code> (WP6) → <code>ThymeleafProjectWorkflowTool</code>
대상	DB 스키마 기반 신규 화면	운영 중인 레거시 JSP 화면 변환
안전장치	Region 3-way(Base·Current·New) 비교 기반 자동 병합	Preview Hash 승인 + source drift 재검증 + 분산락
승인	<code>llmProvider=auto</code> 면 기본적으로 즉시 적용 — 고위험 테이블로 지정된 경우만 승인된 화면명세 필요(2.2절 <code>CrudGenerationApprovalPolicy</code> , 기본값 비활성)	<code>PREVIEW_READY</code> 에서 hash 일치해야만 <code>approve()</code> 통과
공유 지점	<code>ApprovedProjectWritePort</code> · <code>ProjectWritePolicy.ATOMIC_APPROVED</code> (둘 다 사용, 승인 여부는 각자 책임)	
DB 저장	<code>AI_CRUD_GENERATION_SNAPSHOT</code> (51건) + 신규 <code>AI_GENERATION_OPERATION_AUDIT</code> · <code>AI_GENERATION_VALIDATION_EVIDENCE</code> (V16, 14.1절 참고)	
	<code>AI_THYMELEAF_PROJECT_OPERATION</code> (2,991건) + <code>AI_ARTIFACT</code> · <code>AI_OPERATION_EVENT</code> · <code>AI_OPERATION_LOCK</code>	

관련 파일: `tools/ThymeleafProjectWorkflowTool.java`·`tools/ThymeleafBaselineApprovalTool.java`·`tools/ThymeleafBindingGenerationTool.java`·`service/thymeleaf/ThymeleafProjectWorkflowService.java`·`service/thymeleaf/ThymeleafBindingGenerationService.java`·`service/write/ApprovedProjectWritePort.java`·`model/thymeleaf/ProjectOperationStatus.java`. `previewThymeleafBindingGeneration`을 실제로 트리거하는 상위 스킴·프롬프트 흐름은 CLAUDE.md·스킴 문서 어디에도 언급이 없어 **미확인**으로 남습니다 — MCP Tool로는 노출돼 있고 DB에는 실사용 흔적이 뚜렷하지만, 무엇이 이걸 부르는지는 코드만으로 확인할 수 없었습니다.

14.1 실행기 ↔ 공통 제어 계층(Control Plane) 관계도 구현·커밋 완료

`docs/figma/32~34_Dual_Pipeline_Common_Control_Plane_*.md`(DPC-602)의 제안이 실제 코드로 구현되고, 커밋 `5ae8507` "feat: add dual pipeline control plane"과 후속 보완 커밋 `f439db3` "security: avoid storing dashboard credentials"로 커밋까지 완료됐습니다(관련 테스트 19개 전부 통과). 다만 32번 문서 §8의 종료 조건 — `code-reviewer/architect` 독립 검토 증적, 2026-08-25~2026-09-24 최소 30일 Readiness 관측 — 은 아직 진행 중이므로, 이 절의 초록색 배지는 "코드가 커밋된 상태"를 의미할 뿐 "Merge·전환 최종 확정"을 의미하지 않습니다. 두 실행기의 실제 Apply·쓰기 경로 자체는 위 14번 다이어그램과 동일하게 **변경 없음** — 이 관계도는 여전히 **읽기 전용 투영(Projection)** 계층입니다.



— 기존 · 이미 커밋된 코드 — 신규 구현 · 커밋 완료(5ae8507 , 테스트 19/19)

GenerationControlPlaneService는 실행기 내부 로직을 호출하지 않고, 각 실행기가 이미 쓰고 있는 기존 테이블을 읽기 전용으로 투영합니다 (33_..._Specification.md §2 그대로 구현). CrudGenerationSnapshotAdapter가 투영하는 대상은 2.2절 7번 (GenerateCrudProjectUseCase) 내부·14번 다이어그램에 그려진 CodeServiceGenerationExecutor가 실제로 수행하는 **3-way(Base·Current·New) Region 비교** 결과이며, 이번 구현에서 두 가지 기존 한계가 실제로 해소됐습니다. ① **Evidence 문제(해결됨)** — 기존 AI_CRUD_GENERATION_SNAPSHOT에는 검증/Gate 필드가 없었지만(코드 확인 완료), 새 CrudValidationEvidenceRecorder가 GenerationVerifierRunner의 관찰 혹은 통해 RENDER·BUILD·BINDING Gate 결과를 AI_GENERATION_VALIDATION_EVIDENCE(V16)에 실제로 기록합니다. ② **Region Conflict 이력 문제(해결됨)** — CodeServiceGenerationExecutor가 CONFLICT·FAILED·APPLIED 세 종료 경로 모두에서 recordAudit()를 호출해 changedRegionIds·preservedRegionIds·conflictRegionIds를 AI_GENERATION_OPERATION_AUDIT (V16)에 매 시도마다 남기므로, 더 이상 성공 Snapshot의 고정된 mergePolicy=REGENERATE 값에만 의존하지 않습니다. Thymeleaf의 SECURITY Gate는 여전히 현재 코드에 없어 기존 Operation의 필수 조건으로 취급하지 않습니다. 저장 전략은 docs/architecture/ADR_Dual_Pipeline_Legacy_Storage_Adapter_Strategy.md 에서 "기존 두 이력 테이블은 원본 그대로 유지하고, 신규 교차 데이터만 Sidecar 테이블에 별도 기록"하는 방식으로 확정됐습니다. 커밋 5ae8507/f439db3로 코드는 이미 커밋됐지만, 남은 것은 코드가 아니라 절차입니다 — 32번 문서 §8의 종료 조건(① code-reviewer/architect 독립 검토 증적, ② 2026-08-25부터 최소 30일간 AI_GENERATION_READINESS_COMPARISON 관측 후 2026-09-24 이후 분석)이 아직 충족되지 않았으므로 이 절의 "구현·커밋 완료" 표시는 Merge·전환 최종 확정을 의미하지 않습니다.

핵심 소스: README.md, McpConfig.java(Tool 등록·인가), McpToolDefinitionSnapshotTest(105개 Tool 메서드 / 40개 Tool 객체 기준선, 2026-08-28 compareDesignFidelity·downloadFigmaAssets·checkResponsiveRegression 추가 반영), 각 서브 프로젝트 README.md, docs/figma/18_..._Impact_and_Implementation_Specification.md·19_..._Implementation_List.md(Catalog/Registry v1→v2→v3 SSOT 전환), ComponentRegistryDualReadService·KrdsComponentResolutionService 소스(현재 어느 경로가 실제 운영 중인지 import 기준으로 확인), KrdsQnaRuntimeResolverIntegrationTest, docs/figma/05_Overall_Architecture_Diagram.md(WEB_CAPTURE vs Semantic Pipeline 범위 구분), DesignReferenceTool.java·ThymeleafLayoutTool.java(5·6번), jsp-design-extractor/README.md(8번), component-contracts/tokens/**(9번, W3C Design Tokens 형식), krds-design-system-author-plugin/README.md(10번), jsp-to-figma-plugin/src/code.ts(11번), FigmaExportBundle.java·FigmaExportTool.java·FigmaExportController.java(2번 내보내기 경로 및 5번↔2번 연결), ProjectInitializrTool.java·FilePlanFactory.java(gnbLayoutHtml 등)·ThymeleafLayoutTool.java의 Tool 설명 원문(2.2, packageName 일치 요구사항과 overwrite 관계 확인), CrudGenerationDispatchService.java("llmProvider 분기의 유일한 지점" JavaDoc)·CrudScreenSourceTool.java·JoinQueryTool.java(2.2 보조 진입점), EgovOllamaChatController.java·EgovSessionAwareChatServiceImpl.java(2.3, RAG/Chat 흐름), PipelineEvolutionProperties.java·PipelineReleaseGateController.java·PipelineArtifactController.java·PipelineReleaseReadiness.java·ScreenHandoffProjectionService.java(12번), docs/figma/29_5Axis_Benchmark-Based_Pipeline_Evolution_Implementation_Specification.md·30_5Axis_Benchmark-Based_Pipeline_Evolution_Implementation_List.md·31_5Axis_Pipeline_Release_Gate_Operation_근거 문서), src/main/resources/db/migration/V1~V15__*.sql·mapper/*Repository.java(20종)·service/evidence/PreviewEvidenceBundleRepository.java·service/handoff/ScreenHandoffBundleRepository.java·EgovRedisChatMemoryRepository.java(13번, 실제 운영 DB SHOW TABLES 대조로 검증).

- 패키지 세부 구조와 각 서브 프로젝트 역할은 README.md의 "디렉터리 구조" 섹션을 참고하십시오.
- ComponentRegistry.java의 SCHEMA_VERSION 상수는 리터럴 값이 "component-registry-v1"이지만 코드베이스 어디서도 참조되지 않는 죽은 상수입니다 — 실제 필드 구조는 component-registry-v2.schema.json과 일치하고 ComponentRegistryV2ToV3Converter JavaDoc·fixture 파일명도 이를 "v2"로 부르므로, 3번 다이어그램은 그 실무 관례를 따랐습니다.
- 2026-08-23 갱신:** d009058 커밋("feat: complete 5-axis pipeline release gates", 299 files/+22394)을 반영해 12번 다이어그램(Pipeline Evolution — Release Gate와 신규 계약)을 추가했습니다. 1~11번 다이어그램은 이번 갱신에서 구조를 바꾸지 않았고, 2.1의 03단계 설명 한 줄만 12번을 가리키도록 보강했습니다.
- 2026-08-25 갱신:** 13번 다이어그램(MySQL AI_* 테이블 데이터 흐름 — 생성·갱신 계약)을 추가했습니다. Flyway V1~V15가 정의하는 AI_* 테이블 20종을 실제 운영 DB(docker exec egov-mysql mysql -u ebt -pebt01 ebt -e "SHOW TABLES")와 대조해 코드·DB 드리프트가 없음을 확인했고, WORM/Revision-append/Upsert 3가지 영속 계약으로 분류했습니다. 조사 중 12번 다이어그램의 PreviewEvidenceBundle·ScreenHandoffBundle이 실제로는 MySQL이 아니라 ConcurrentHashMap 인메모리라는 점을 발견해 13번 섹션에 별도 발견 박스로 표시했습니다 — 12번 다이어그램 자체는 이번 갱신에서 구조를 바꾸지 않았습니다.
- 2026-08-25 갱신(2):** 14번 다이어그램(Thymeleaf 마이그레이션 승인 워크플로우 — CRUD 소스 생성과의 승인 단계 비교)을 추가했습니다. ThymeleafProjectWorkflowService의 상태기계를 실제 운영 DB 실측치(AI_THYMELEAF_PROJECT_OPERATION 상태별 COUNT(*), PREVIEW_READY 1282·APPROVED 838·APPLIED 703·CONFLICT 168)로 검증했고, ApprovedProjectWritePort javadoc을 근거로 이 워크플로우와 2.2절 CRUD 소스 생성 경로가 파일 쓰기 계층은 설계 시점부터 공유하되 승인 모델은 이원화되어 있다는 점을 핵심 결론으로 정리했습니다.
- 2026-08-25 갱신(3):** 14.1절(실행기 ↔ 공통 제어 계층 관계도)을 커밋 5ae8507/f439db3 기준으로 검증해 "제안·미구현"에서 "구현·커밋 완료"로 갱신했습니다. Flyway V2·V15·V16·V18이 정의하는 실제 컬럼을 다이어그램 박스 안에 반영하고, 13번 표에 신규 테이블 3종을 추가했으며, "2.2절"이 실제로는 다이어그램에 없는 CodeServiceGenerationExecutor를 가리키던 인용 오류를 "2.2절 7번(GenerateCrudProjectUseCase) 내부"로 정정했습니다. 다만 32번 문서 §8의 독립 검토·30일 Readiness 관측 조건은 아직 충족되지 않아 Merge·전환 최종 확정과는 구분됩니다.
- 2026-08-26 갱신:** 2.2절 다이어그램에 CrudGenerationApprovalPolicy(신규) 박스를 GenerateCrudProjectUseCase(7번)와 JoinQueryTool 사이에 새로 추가했습니다(커밋 5b8244a). 기존 V2_APPLY+Thymeleaf 화면명세 필수화 가드와 별개로, 설정된 고위험 테이블(또는 전체)은 viewType 무관하게 승인된 화면명세 없이는 생성이 차단되며 AI_GENERATION_OPERATION_AUDIT에 기록됩니다. 검토·설계 과정에서 Thymeleaf식 3단계 구조(옵션 A)도 검토했으나 기존 Tool 계약을 깨는 변경이라 채택하지 않았습니다 — 근거는 docs/architecture/CRUD_명시적_승인_단계_도입_타당성_검토.md 참고. (최초 반영 시 이 박스를 서브텍스트로만 표시했다가, "너무 시시하다"는 지적을 받아 전용 박스로 승격했습니다 — JoinQueryTool/MenuTool 위치는 아래로 밀려 이동했습니다.)
- 2026-08-26 갱신(2):** ApprovedProjectWritePort가 6번 다이어그램(Thymeleaf 생성 서비스 상세) 1~5번 중 어디에 속하는지 검토하다가, 4·5번 박스(CodeTemplateTool·CodeSaverTool.saveCode())가 실제로는 llmProvider=auto 경로가 아니라 claude 수동 경로의 Tool이라는 사실 오류를 발견해 정정했습니다 — CodeTemplateTool은 코드 주석에 "디버깅·확인 용도"로 명시돼 있고, CodeSaverTool.saveCode()는 ApprovedProjectWritePort를 전혀 쓰지 않습니다. 3단계(CrudModelFactory.fromSchema()) 이후를 CrudGenerationDispatchService.isAuto() 기준 auto/claude 두 갈래로 나눠 다시 그렸습니다 — auto 경로는 CrudGenerationRenderer(내부 CrudTemplateRenderer)로 서버가 직접 렌더링한 뒤 CodeServiceGenerationExecutor가 ApprovedProjectWritePort로 저장합니다(14번 다이어그램의 그 Port와 동일 인스턴스 계약). 결론적으로 ApprovedProjectWritePort는 6번 다이어그램의 기존 1~5번 어디에도 속하지 않았고, 새로 추가한 auto 경로의 5a번 박스가 그 자리입니다.
- 2026-08-26 갱신(3):** 12번 다이어그램("Pipeline Evolution")에서 "파이프라인"을 특정 대상 없이 써서 이 문서에 여러 파이프라인(2번의 목표 화면 생성 파이프라인, 4번의 6단계 Q&A Resolver 파이프라인 등)이 있는데 어느 것인지 알 수 없다는 지적을 받아, h2 제목과 본문·범례·배지 서브텍스트를 모두 "목표 화면 생성 파이프라인"(2번 다이어그램 2.1절에서 정의한 S1~S7 파이프라인 — 이 문서에서 S-번호를 쓰는 유일한 파이프라인)으로 명시했습니다. 그 과정에서 S3 박스의 점프 배지는 "2"(2번 다이어그램)를 가리키면서 바로 위 서브텍스트는 "1번 파이프라인"이라고 쓴 자기모순도 함께 발견해 정정했습니다.
- 2026-08-26 갱신(4):** 같은 방식으로 4번 다이어그램("Q&A Runtime Resolver 상세")도 검토했습니다. 화면에 보이는 텍스트는 이미 이름이 있었지만 스크린리더용 aria-label이 "6단계 파이프라인"이라고만 써서 어느 파이프라인인지 특정하지 않아 "Q&A Runtime Resolver 파이프라인"으로 명시했습니다. 4번은 다른 다이어그램으로 가는 점프 링크가 없어 12번에서 발견한 것과 같은 배지-텍스트 자기모순은 없었습니다.
- 2026-08-26 갱신(5):** 4번·12번에서 발견한 두 가지 문제(이름 없는 "파이프라인" 참조, 점프 배지-서브텍스트 번호 자기모순)를 나머지 전체 다이어그램(1·2·3·5·8·9·10·11·13번)에 대해서도 점검했습니다. jump-badge 숫자와 그 링크가 실제로 가리키는 섹션의 h2 번호를 스크립트로 전수 대조한 결과 추가 자기모순은 없었습니다. "파이프라인"이라는 단어는 13번 다이어그램에서 2곳 더 이름 없이 쓰이고 있어("실제 파이프라인 호출 경로", "CRUD 소스 생성 파이프라인 호출부") — 이걸 이 문서의 두 기명 파이프라인(목표 화면 생성 파이프라인 / Q&A Runtime Resolver 파이프라인) 중 어느 쪽도 아니어서, 새 이름을 지어내는 대신 "파이프라인"을 "흐름"으로 바꿔 불필요한 세 번째 파이프라인처럼 보이지 않게 정정했습니다. 12번 다이어그램 범례(legend)에도 "기존 파이프라인 앵커"로 남아 있던 문구 1곳을 12번 본문과 동일하게 "목표 화면 생성 파이프라인 앵커"로 통일했습니다. 1·2·3·5·8·9·10·11번 다이어그램 자체는 이름·번호 참조 모두 문제없이 정확했습니다.
- 2026-08-26 갱신(6):** 2.2절 3단계의 BoardGenerationTool 박스 밑 화살표가 BoardGenerationTool 전용인지, 화살표 없는 MasterDetailGenerationTool은 거기서 끝나는 것인지 질문을 받아 실제 소스를 확인한 결과 **다이어그램 오류**였습니다. CrudGenerationTool→CrudGenerationDispatchService, MasterDetailGenerationTool→MasterDetailGenerationDispatchService(이름은 비슷하지만 완전히 별개 클래스, 둘 다 자체 isAuto() 분기 보유),

- BoardGenerationTool→GenerateBoardProjectUseCase(분기 자체 없음, llmProvider 파라미터도 없음) — 이렇게 세 갈래가 서로 다른데, 기존 다이어그램은 단일 화살표 하나를 가운데 열(Board 박스 바로 밑)에 그려놔서 세 Tool 중 어느 것도 정확히 설명하지 못했습니다. 세 Tool 각각 밑에 실제 대상 박스를 따로 그리도록 재작성했고, CRUD의 isAuto() auto/claude 분기·CrudGenerationApprovalPolicy·JoinQueryTool·MenuTool 체인은 위치만 왼쪽으로 옮겼을 뿐 내용은 그대로입니다. 범례의 "auto 경로 ... 유일한 지점"이라는 문구도 CRUD 전용이라는 오해를 줄 수 있어 "CRUD auto 경로"로 한정했습니다.
- **2026-08-26 갱신(7):** 2.2절 다이어그램에서 layoutMode=reuse 기본값 옛지 라벨이 "layoutM"까지만 보이고 잘린다는 스크린샷 지적을 받아 확인한 결과, 라벨 텍스트(x=600 y=380, 좌측 정렬)가 바로 다음에 그려지는 CrudScreenSourceTool 등 (대안) 박스(x=640~1140)와 겹쳐서 SVG 문서 순서상 뒤에 그려지는 박스가 텍스트 뒷부분을 그대로 덮어버리고 있었습니다(사용자 캡처 크롭 문제가 아니라 실제 렌더링 버그). 라벨을 두 대안 박스 위쪽 빈 공간(y=300, 세로선 기준 가운데 정렬)으로 옮겨 겹침을 없앴습니다.
 - **2026-08-26 갱신(8):** 세 Tool의 "저장 주체·저장 내용"을 비교 분석하다가, BoardGenerationPipelineService·MasterDetailGenerationPipelineService가 둘 다 GenerationExecutor 인터페이스를 주입받는데 그 구현체가 코드베이스 전체에서 CodeServiceGenerationExecutor 단 하나뿐이라는 사실을 확인했습니다 — 분기(Dispatch) 로직은 세 Tool이 서로 다르지만(2.2절 위 세 박스), 실제로 파일을 쓰는 최종 실행기·Write Port는 Spring 싱글턴으로 동일합니다. 세 종점 박스 아래에 CodeServiceGenerationExecutor(공통 실행기) 합류 박스를 추가하고, CRUD·Board는 항상·MasterDetail은 auto 경로만 이 박스로 모이도록 화살표를 그렸습니다. 도메인별 차이는 저장 주체가 아니라 각자 다른 Renderer(CrudTemplateRenderer/BoardTemplateRenderer/MasterDetailTemplateRenderer)가 만드는 파일 내용에 있습니다.
 - **2026-08-26 갱신(9):** 2.2절 레이아웃을 정리했습니다 — GenerateCrudProjectUseCase·CrudGenerationApprovalPolicy·JoinQueryTool을 CrudGenerationDispatchService(1열)에 맞춰 왼쪽으로 재정렬하고, CodeServiceGenerationExecutor(공통 실행기) 박스는 가운데 열 쪽으로 옮겼습니다(자식 박스와의 겹침을 피하려 정확히 중앙은 아님). 세 Tool의 "저장 주체·저장 내용" 비교표를 새로 추가해 분기(Dispatch)·auto 저장 주체·claude 경로·렌더러·레이어 정의·실제 파일 쓰기·이력 기록을 한눈에 대조할 수 있게 했습니다 — 실제 파일 쓰기·이력 기록 행은 세 Tool이 동일해서 병합 표시했습니다.
 - **2026-08-26 갱신(10):** GenerateCrudProjectUseCase 박스 안 "서버가 직접 디스크에 파일 저장 → AI_GENERATION_HISTORY 기록" 문구가 바로 아래 CodeServiceGenerationExecutor (공통 실행기) 박스와 내용이 겹치는 것 아니냐는 지적을 받아 CrudGenerationApplicationService .execute() 소스로 확인했습니다 — 둘 다 정확하고, 이중 저장이 아니라 같은 메서드 호출 하나(planner.plan()→renderer.render()→executor.execute()→historyRecorder.record())를 서로 다른 확대 수준에서 설명한 것입니다. CrudGenerationApprovalPolicy와 동일한 "내부 호출" 관계라, 화살표에 라벨을 추가하고 figcaption에도 명시적으로 정리했습니다.
 - **2026-08-26 갱신(11):** CrudGenerationApprovalPolicy와 CodeServiceGenerationExecutor의 분기 기준을 물어봐서 CrudGenerationPlanner .plan() 소스로 정확한 조건(requiresApproval(tableName) && 두 ID 모두 비어있음 → 그 자리에서 REJECTED, executor 미호출)을 확인하고 다이어그램에 반영했습니다. 기존엔 CodeServiceGenerationExecutor로 가는 화살표가 GenerateCrudProjectUseCase에서 곧장 나와 ApprovalPolicy를 건너뛰는 것처럼 보였는데, 이제 ApprovalPolicy를 통과한 뒤에 이어지도록 화살표를 다시 그렸고, 박스 안에 "차단 시 executor 미호출" 문구와 별도 figcaption을 추가해 이게 두 갈래 분기가 아니라 단일 순차 게이트임을 명시했습니다.
 - **2026-08-26 갱신(12):** CrudGenerationApprovalPolicy→CodeServiceGenerationExecutor 화살표 라벨 "내부 호출(같은 저장 동작, 별도 저장 아님)"이 무슨 뜻인지 모르겠다는 지적을 받아 "승인 통과 → 이어서 저장 실행"으로 쉬운 말로 바꿨습니다 — 자세한 기술적 설명(같은 execute() 메서드 안에서 순서대로 호출된다는 점)은 라벨 대신 옆 figcaption에 그대로 남겨뒀습니다.
 - **2026-08-26 갱신(13):** GenerateCrudProjectUseCase 박스의 "서버가 직접 디스크에 파일 저장" 문구가 무조건 일어나는 것처럼 보인다는 지적을 받아 "승인 통과 시 서버가 직접 디스크에 파일 저장"으로 조건을 박스 텍스트 자체에 명시했습니다 — 승인이 필요한 고위험 테이블 인데 화면명세가 없으면 이 저장 자체가 일어나지 않는다는 점(아래 CrudGenerationApprovalPolicy 박스 참고)을 화살표만 보지 않고도 알 수 있게 했습니다.
 - **2026-08-26 갱신(14):** "서버가"처럼 서버명을 명시하지 않은 표현과, "파일 저장"이라고만 쓰고 무슨 내용을 저장하는지 안 밝힌 곳을 2.2절·6번 다이어그램에서 정리했습니다. 모든 "서버"를 2.1절 박스 명칭과 통일해 **"Spring MCP 서버"**로 명시했고, "파일 저장"은 각 도메인의 실제 저장 내용으로 구체화했습니다 — CRUD는 "CRUD 소스 파일 세트"(VO/Controller/Service/Mapper/View), Board는 "게시판(BBS) 소스 파일 세트", MasterDetail은 "마스터·디테일 소스 파일 세트"입니다. GenerateBoardProjectUseCase·MasterDetailGenerationDispatchService·GenerateCrudProjectUseCase 박스 높이를 늘려 문구를 담았고, 그 아래 CrudGenerationApprovalPolicy·JoinQueryTool·MenuTool 박스와 연결선을 전부 같은 폭 만큼 아래로 밀어 겹침을 피했습니다(viewBox 높이도 1180→1200으로 늘림).
 - **2026-08-26 갱신(15):** "승인 통과 → 이어서 저장 실행" 다음에 CodeServiceGenerationExecutor가 정확히 뭘 저장하는지 검토해달라는 요청을 받아 CrudLayerDefinition 소스로 확인했습니다. 파일을 하나씩이 아니라 ProjectChangeSet 하나로 묶어 원자적으로 저장하며, CRUD 기준 공통 7개(VO·Mapper·Mapper.xml·Service·ServiceImpl·Controller·ValidationHandler) + JSP 전용 4개(총 11개) 또는 Thymeleaf 전용 9개(총 16개)라는 정확한 레이어 목록·개수를 확인해 실행기 박스와 별도 figcaption에 반영했습니다 — "JSP 11개/Thymeleaf 16개" 수치는 CLAUDE.md 기준 기재와 일치합니다.
 - **2026-08-27 갱신:** 2.2절 기존 표(분기·렌더러·저장 주체 비교)는 "누가/무엇을 저장하는가"만 다루고 있어, **Tool 호출 계약**(역할 슬롯 수·테이블명 자유도·llmProvider 파라미터 존재 여부·반환 타입) 관점의 비교표를 그 아래에 추가했습니다. 실제 코드(CrudGenerationTool·MasterDetailGenerationTool·BoardGenerationTool의 파라미터 시그니처, BoardTableSetResolver의 fallback 로직)로 확인한 내용이며, BoardGenerationTool의 mainTable 등 테이블명 파라미터가 LETTNBBS 등으로 하드코딩돼 있다는 오해가 있을 수 있어 "고정된 것은 테이블 이름이 아니라 역할 슬롯 개수"라는 점을 명시했습니다.
 - **2026-08-27 갱신(2):** "auto/claude 분기"가 정확히 무엇을 뜻하는지 질문을 받아, 위 비교표 바로 아래에 auto/claude 각각의 실행 주체·저장 여부·토론 소비·반환 타입을 대조하는 표와 설명을 추가했습니다. 이 분기는 Tool마다 서로 다른 서비스(CrudGenerationDispatchService/MasterDetailGenerationDispatchService)가 각자 isAuto()로 수행하며, BoardGenerationTool은 이 스위치 자체가 없다는 점을 다시 명확히 했습니다.
 - **2026-08-27 갱신(3) — 클래스명 정정:** 2.2절 전체에서 CRUD 진입점으로 표기하던 CrudPromptBuilderTool이 실제로는 **MCP에 전혀 등록되지 않는 클래스**임을 확인했습니다 — @Component/@Service 어노테이션이 없고, McpConfig.allToolCallbacks(...)에도 등장하지 않으며, 무엇보다 @McpToolRisk가 없어서(등록 시도 시 기동 자체가 즉시 차단되는 구조) 처음부터 등록될 수 없는 상태입니다. 실제 등록된 CRUD 진입점은 tools/generation/ CrudGenerationTool.java(@Component, @McpToolRisk(FILE_WRITE) 있음)이며, 같은 CrudGenerationMcpFacade로 위임하므로 동작 로직 자체는 이전 서술과 동일합니다. 이번 갱신에서 2.2절 전체의 CrudPromptBuilderTool 표기를 CrudGenerationTool로 교체했습니다. **(2026-08-27 후속 갱신)** CrudPromptBuilderTool.java와 그 전용 테스트는 이후 실제로 삭제됐고, 테스트 커버리지는 실제 등록된 6개 클래스(CrudGenerationTool·MasterDetailGenerationTool·BoardGenerationTool·CrudScreenSourceTool·BoardScreenSourceTool·MasterDetailScreenSourceTool) 테스트로 이관됐습니다.
 - **2026-08-27 갱신(4):** 5번 다이어그램(DesignReferenceTool)의 "UiDesignSpec은 시각 구조·Token·Component 선택만 공급한다"는 서술이 색상/컴포넌트 스타일도 어느 정도 전달되는 것처럼 읽힐 수 있어, FigmaDesignSpecMapper(1단계 파싱) → ScreenSpecAssembler.assemble() (2단계 매핑) 두 단계를 코드로 추적해 표로 정리했습니다. 결론: 컴포넌트별 색상은 1단계부터 캡처가 안 되고, 화면 전체 배경색·폰트만 1단계에서 캡처되지만 2단계 매핑 코드 부재로 결국 버려집니다 — ScreenSpecification에는 color/token/style 필드가 전혀 없습니다.
 - **2026-08-27 갱신(5):** 6번 다이어그램의 3번 박스(CrudModelFactory.fromSchema())가 isAuto() 분기 이전 공통 단계처럼 그려져 있던 것을 정정했습니다 — 실제로는 auto 경로만 이 클래스를 타고, claude 경로는 완전히 별개인 CrudPromptBuilderService + ScreenSpecificationPromptFormatter로 갑니다. 두 경로가 같은 ScreenSpecification을 소비하는 범위 차이(auto=밀도+필드선택만 기계적 반영, claude=archetype·조인·필드라벨까지 텍스트로 전달)를 비교표로 추가했습니다.
 - **2026-08-27 갱신(6):** 갱신(4)에서 표로만 정리했던 색상·토큰 소실 내용을 5번 다이어그램의 플로우 자체에도 반영해달라는 요청을 받아, "1단계·FigmaDesignSpecMapper 파싱"과 "2단계·ScreenSpecAssembler.assemble()" 두 박스를 실제 흐름 아래에 점선으로 추가했습니다. 각 박스는 무엇이 캡처/전달되고 무엇이 버려지는지를 다이어그램에서 바로 보여주며, 범례에 "시각 정보 소실 지점"(rest 색상) 항목을 추가했습니다. 표 형태의 상세 근거(파일·라인)는 갱신(4)의 비교표에 그대로 유지됩니다.

- **2026-08-28 갱신:** ① BoardGenerationTool에 llmProvider(auto/claude) 분기가 추가됐습니다(BoardGenerationDispatchService/BoardPromptGenerationService/ BoardToolResult 신규, BoardGenerationTool_llmProvider_구현계획.md) — 2.2절의 "Board는 분기 자체가 없다"는 서술 전부(다이어그램 박스·비교표 2개·본문 3곳)를 "세 Tool 모두 각자 별개의 isAuto() 분기를 갖는다"로 정정했습니다(2026-08-26 갱신(6) 항목은 **그 시점 기준 정확했던 역사적 기록이라 그대로 뒀습니다**). ② 갱신(4)·(6)에서 정리한 "색상·토큰 2단계 소실"이 Figma_fills_strokes_구현계획.md(커밋 bbc8079)·Figma_픽셀재현_claude경로_구현계획.md(커밋 369cbe1)로 부분 해소됐습니다 — componentStyles(색상)·componentGeometry(좌표·auto-layout·텍스트스타일)가 이제 ScreenSpecification까지 매핑되고 claude 경로 프롬프트에 반영되지만, auto 경로는 여전히 미참조하고 루트 배경색·폰트(tokens)는 지금도 안 옮겨집니다 — 5·6번 다이어그램의 박스·표·범례를 이 최신 상태로 다시 정정했습니다. ③ 신규 MCP Tool 2종(compareDesignFidelity — 구조적 일치도 측정, downloadFigmaAssets — 이미지 asset 다운로드, 둘 다 Figma_픽셀재현_제외범위_구현계획.md)을 5번 섹션에 추가했습니다. ④ Tool 총계를 104개 메서드/39개 객체로 갱신했습니다. auto 경로 CSS 색상 자동주입은 "고정 KRDS 체계 유지"라는 이 프로젝트 전제와 모순되어 **기각**됐고(Figma_색상_CSS자동주입_ auto경로_검토.md), 게시판 DASHBOARD 화면 생성 지원은 검토 후 **보류** 상태입니다 — 둘 다 이 다이어그램에는 반영할 신규 아키텍처가 없어 별도 박스를 추가하지 않았습니다.
- **2026-08-28 갱신(2):** ① 갱신(1) ②에서 "지금도 안 옮겨진다"고 적었던 루트 배경색·폰트(tokens)가 ScreenSpecAssembler.assemble() 매핑(커밋 4a01814)과 ScreenSpecificationPromptFormatter 프롬프트 반영(커밋 a1c1e30)까지 마저 완료돼, 이제 componentStyles/componentGeometry와 동일하게 claude 경로 끝까지 생존합니다(auto 경로 미참조는 그대로) — 5번 다이어그램의 박스·표·범례·문단을 이 최신 상태로 다시 정정했습니다. ② Figma_픽셀재현_제외범위_구현계획.md 트랙 C의 잔여 리스크(반응형 가드레일 문구를 실제로 지켰는지 확인할 자동화 수단 없음)를 해소하는 checkResponsiveRegression Tool을 추가했습니다(Figma_픽셀재현_2차구현_ 반응형검증_구현계획.md, 커밋 7d543e9) — 신규 캡처 로직 없이 captureWebPageMultiViewport()가 이미 만드는 breakpointObservations를 재분류해 MOVED를 가드레일 위반 의심 신호로 요약합니다. ③ Tool 총계를 105개 메서드/40개 객체로 갱신했습니다.
- 이 다이어그램은 정적 스냅샷입니다 — Tool·서비스 구성이 바뀌면 함께 갱신이 필요합니다.